



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

Konzeption und Implementierung einer kontexterweiterten Aggregationsinfrastruktur zur kollaborativen Anomalieerkennung

Forschungsarbeit

von

Yannic Hanel

3296954

im Rahmen des Studiums Informatik (M.Sc.)

an der

Ostbayerischen Technischen Hochschule Regensburg
Fakultät für Informatik und Mathematik

Betreuer: Prof. Dr. Sebastian Fischer

Regensburg, 27.05.2026

Inhaltsverzeichnis

Abkürzungsverzeichnis	4
1 Einleitung	1
1.1 Motivation und Problemstellung	1
1.2 Zielsetzung der Arbeit	1
1.3 Aufbau der Arbeit	1
2 Stand der Forschung	3
2.1 Anomalieerkennung und Alert-Aggregation in verteilten Systemen	3
2.2 Informationsfusion mit variabler Eingabegröße und Attention-Mechanismen . . .	3
2.3 Kollaboratives Threat-Report-Sharing, Gewichtungsstrategien und Datenqualität .	4
3 Systemarchitektur und Detektion Framework	5
3.1 Systemüberblick und verteilte Kommunikation	5
3.1.1 Privates IPFS-Netzwerk und Peer-to-Peer-Datenaustausch	6
3.1.2 End-to-End-Datenfluss	6
3.2 Physische Testumgebung	8
3.3 Detektionseinheiten der Ladeinfrastruktur	9
3.3.1 System- und Leistungsanomaliedetektion	10
3.3.2 Physikalische Signal- und Temperaturüberwachung	10
3.3.3 KI-basierte Angriffserkennung im Netzwerk	11
3.3.4 Hardware-Ereignisüberwachung	11
4 Konzeption des Threat Reporters	12
4.1 Zentralisiertes Datenmanagement und Merkmalsextraktion für den Threat-Report	12
4.2 Zeitliche Diskretisierung und Intervalldefinition	12
4.3 Statistische Merkmalsaggregation mittels Sliding-Window-Analyse	13
5 Theoretische Grundlagen der Informationsfusion	14
5.1 Vergleich von Aggregationsmethoden	14
5.1.1 Statische Aggregation	15
5.1.2 Padding und Masking	16
5.1.3 Deep Sets	16
5.1.4 Set2Set	17
5.1.5 Graph Neural Networks und Pooling	17
6 Methodik: Gewichtete Aggregation von Threat Reports	18
6.1 Anforderungen an die Gewichtungsfunktion	18
6.2 Auswahl der Gewichtungsfunktion	19
6.2.1 Utility-Theory-Funktion	19
6.2.2 Sigmoid-Funktion	19
6.3 Implementierung des Attention-basierten Selektionsmodells	20

7 Funktionsverifikation	22
7.1 Versuchsaufbau	22
7.2 Szenariobasierte Funktionsvalidierung	23
7.2.1 Szenario 1 – NaN-Werte (Fehlerbehandlung)	23
7.2.2 Szenario 2 – Verschobene Zeitreihe	24
7.2.3 Szenario 3 – Uniforme negative Detektionen	24
7.2.4 Szenario 4 – Ausreißer-Report	26
7.3 Analyse des Echo-Chamber-Effekts	27
7.3.1 Definition und theoretischer Kontext	27
7.3.2 Formalisierung des Szenarios	27
7.3.3 Einordnung und Ausblick	28
8 Diskussion und Forschungslücken	29
8.1 Auswirkungen der Gewichtungsstrategien auf die Threat-Report-Qualität	29
8.2 Potenzial und Grenzen der Aggregation für die Detektionsqualität	30
8.2.1 Datenqualität vs. Datenmenge	30
8.2.2 Abtastintervall	30
8.2.3 Verschobene Zeitfenster	30
8.2.4 Echo-Chamber-Effekt	30
8.2.5 Evaluation auf öffentlichem Datensatz	31
9 Zusammenfassung und Ausblick	32
9.1 Methodik und Implementierung	32
9.2 Ergebnisse der Funktionsverifikation	32
9.3 Zukünftige Forschungsaspekte	32
10 Quellcode	34
Literaturverzeichnis	46

Abkürzungsverzeichnis

DNN	Deep Neural Network
DoS	Denial of Service
DDoS	Distributed Denial of Service
IPFS	Inter Planetary File System
P2P	Peer-to-Peer
CID	Content Identifier
MADDR	Multiaddress
I2C	Inter-Integrated Circuit
MSE	Mean Squared Error
LSTM	Long Short Term Memory
GAT	Graph Attention Networks
NaN	Not a Number
SOC	Security Operations Center
CARA	Constant Absolute Risk Aversion
IoT	Internet of Things
LLM	Large Language Model
ML	Machine Learning
CSV	Comma-separated values
JSON	JavaScript Object Notation
kB	Kilobyte

1 Einleitung

Die folgenden Abschnitte führen in die konkrete Problemstellung, die Zielsetzung sowie den strukturellen Aufbau der Arbeit ein.

1.1 Motivation und Problemstellung

Obwohl die knotenbasierten Detektionsmechanismen der betrachteten Ladeinfrastruktur bereits eine beachtliche Erkennungsleistung aufweisen (siehe Abbildung 7.2), besteht das Ziel, die Spezifität (True-Negative-Rate) sowie die Robustheit des Gesamtsystems weiter zu steigern. Lokale Anomalieerkennungsverfahren stoßen systembedingt an Grenzen, wenn netzwerkweite Veränderungen oder subtile, koordinierte Manipulationsversuche vorliegen [9], die auf Ebene einer einzelnen Wallbox nicht als bösartig identifiziert werden können.

Die Motivation dieser Arbeit liegt daher in der Entwicklung einer Methodik, die es einzelnen Ladepunkten ermöglicht, Kontextinformationen benachbarter Knoten in den Entscheidungsprozess einzubeziehen. Es wird vermutet, dass durch diesen kollaborativen Ansatz räumliche und zeitliche Korrelationen innerhalb des Netzwerks besser erfasst werden können, die über die isolierte Betrachtung eines einzelnen Knotens hinausgehen. Durch die Integration dieser Kontextinformationen wird erhofft, eine präzisere Differenzierung zwischen globalen Schwankungen und lokalen Sicherheitsvorfällen zu ermöglichen. Es wird erwartet, dass dieser erweiterte Kontextbezug zu einer Reduktion von Fehlalarmen (False Positive) sowie einer verbesserten Erkennungsrate tatsächlicher Bedrohungen (True Positive) beiträgt. Inwiefern diese Annahmen empirisch belastbar sind, bleibt jedoch offen und soll in einer weiterführenden Arbeit durch die Evaluation anhand eines unabhängigen Datensatzes untersucht werden.

1.2 Zielsetzung der Arbeit

Um die dargelegte Problemstellung zu adressieren, liegt der Fokus dieser Arbeit auf der Konzeption und Implementierung einer robusten Infrastruktur für die Anomalieerkennung innerhalb der Ladeinfrastruktur. Übergeordnetes Ziel ist die Schaffung einer stabilen Betriebsumgebung, die einen zuverlässigen, kontextbasierten Datenaustausch zwischen den einzelnen Knoten gewährleistet.

Ein wesentlicher Aspekt hierbei ist die Realisierung einer konsistenten und effizienten Aggregation dezentraler Informationen (siehe Kapitel 7 (Funktionsverifikation)), um die Detektionsgüte der beteiligten Entitäten signifikant zu steigern. Nur durch eine performante Umgebung, welche den reibungslosen Austausch von Kontextinformationen sicherstellt, kann die in Kapitel 3 (Systemarchitektur und Detektion Framework) detaillierte Anomalieerkennung erfolgreich implementiert und langfristig wartbar betrieben werden. Die Entwicklung dieser stabilen Kommunikations- und Verarbeitungsarchitektur bildet somit den zentralen Bestandteil der vorliegenden Arbeit.

1.3 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich wie folgt: **Kapitel 2** gibt einen Überblick über den **Stand der Forschung**. Es werden verwandte Arbeiten aus drei thematischen Bereichen diskutiert: erstens

Ansätze zur Anomalieerkennung und Alert-Aggregation in verteilten Systemen, zweitens Methoden der Informationsfusion mit variabler Eingabegröße unter Einsatz von Attention-Mechanismen sowie drittens kollaborative Sharing-Strategien, Gewichtungsverfahren und Fragen der Datenqualität. Die vorgestellten Arbeiten werden jeweils in Bezug zur eigenen Lösung gesetzt und grenzen den Beitrag dieser Arbeit gegenüber dem bestehenden Forschungsstand ab.

Daraufhin wird in **Kapitel 3** die **Systemarchitektur** des ReSiLENT-Systems dargestellt. Der Fokus liegt dabei auf der dezentralen Kommunikation via IPFS sowie den vier Detektionseinheiten, welche physikalische, systemnahe und netzwerkbasierte Anomalien auf Wallbox-Ebene erfassen.

Kapitel 4 beschreibt die **Konzeption des Threat Reporters**. Es wird erläutert, wie Detektionsergebnisse unter Anwendung zeitlicher Diskretisierung und Sliding-Window-Analysen in einem standardisierten CSV-Format aggregiert und im Peer-to-Peer-Netzwerk verteilt werden.

In **Kapitel 5** werden die **theoretischen Grundlagen der Informationsfusion** adressiert. Angesichts variabler Eingabemengen wird die methodische Wahl einer Kombination aus *Deep Sets* zur Gewährleistung der Unverändertheit des Ergebnisses auch bei Vertauschung von Elementen (Permutationsinvarianz) und *Attention*-Mechanismen zur dynamischen Relevanzbewertung begründet.

Darauf aufbauend stellt **Kapitel 6** die **Methodik der gewichteten Aggregation** vor. Hierbei werden die *Negative Exponential Utility Function* sowie eine *logistische Sigmoid-Funktion* hinsichtlich ihres asymptotischen Sättigungsverhaltens und ihrer Robustheit gegenüber statistischen Ausreißern evaluiert.

Die **Implementierung** und Integration dieser Modelle in die ReSiLENT-Architektur werden in **Kapitel 7** beschrieben. Die resultierenden Aggregierten-Threat-Reports werden präsentiert und hinsichtlich ihrer Qualität für die knoteninterne Nutzung bewertet.

Absolvierend erfolgt in **Kapitel 8** eine **Diskussion der Ergebnisse**. Dabei werden kritische Forschungslücken identifiziert, insbesondere die empirische Validierung der Gewichtungsstrategien, das Potenzial LLM-gestützter Aufbereitung für SOC-Operatoren sowie die Untersuchung systemischer Risiken wie des *Echo-Chamber-Effekts*.

Kapitel 9 schließt die Arbeit mit einer **Zusammenfassung und einem Ausblick** ab. Es führt die Kernaspekte der entwickelten Aggregationsinfrastruktur zusammen und zeigt zukünftige Entwicklungs- und Forschungsrichtungen im Bereich der kollaborativen Anomalieerkennung auf.

2 Stand der Forschung

Die Erhöhung der Resilienz dezentraler Ladeinfrastrukturen baut auf bestehenden Erkenntnissen der verteilten Angriffserkennung und Informationsfusion auf. Die folgenden Abschnitte fassen den aktuellen Stand der Forschung in den Bereichen der Alert-Aggregation, der permutationsinvarianten Mengenverarbeitung sowie kollaborativer Sharing-Strategien zusammen.

2.1 Anomalieerkennung und Alert-Aggregation in verteilten Systemen

Lokale Detektionsverfahren stoßen strukturell an Grenzen, wenn netzwerkweite oder koordinierte Angriffe vorliegen, die auf Ebene eines einzelnen Knotens nicht als böartig identifiziert werden können. Lo et al. adressieren dieses Problem mit E-GraphSAGE [9], einem GNN-basierten IDS für IoT-Netzwerke, das topologische Informationen und Kanten-Features aus Netzwerkflussdaten für die Angriffserkennung kombiniert. Heigl et al. präsentieren mit dem SOAAPR-Framework [6] einen Ansatz, der Ausreißererkennungsergebnisse auf Streaming-Daten auswertet und daraus neuartige Angriffsmuster extrahiert. Boswell et al. schlagen mit FLARE [1] eine leichtgewichtige Feature-Aggregationstechnik für IoT-IDS vor, die mittels zeitbasierter Sliding Windows statistische Kennzahlen wie Flussrate, Paketgrößen und Entropie aggregiert, um kurzlebige Angriffsmuster wie DoS und DDoS zuverlässig zu erkennen.

Auf Ebene der Alert-Konsolidierung beschreiben Debar & Wespi [3] einen grundlegenden Aggregations- und Korrelationsalgorithmus, der Duplikate und Folgewarnungen erkennt sowie Alerts situationsbezogen zusammenfasst. Zhang et al. [19] führen diesen Gedanken mit dem IACF-Framework fort, indem sie Alarme basierend auf ihren inneren Zusammenhängen als *Intrusion Actions* modellieren. Durch den Einsatz einer zeitbasierten Sequenzanalyse lassen sich so komplexe, mehrstufige Angriffsmuster sowohl rekonstruieren als auch deren künftigen Verlauf vorhersagen. Im Gegensatz zu diesen regelbasierten Verfahren verfolgt die vorliegende Arbeit einen lernbasierten, gewichteten Aggregationsansatz, der die Relevanz einzelner Threat-Reports dynamisch auf Basis enthaltener Einzeldetektionen bewertet.

2.2 Informationsfusion mit variabler Eingabegröße und Attention-Mechanismen

Eine zentrale Herausforderung bei der Aggregation verteilter Threat-Reports ist die variable Anzahl eingehender Quellen, da klassische neuronale Netze (Fixed-Size Input) erfordern. Zaheer et al. lösen dieses Problem mit der formalen Charakterisierung permutationsinvarianter Funktionen auf Mengen (Deep Sets) [18]: Jede solche Funktion lässt sich als Summen-Dekomposition darstellen, wobei Sölch et al. empirisch zeigen [13], dass die Wahl der Aggregationsfunktion die Modellperformance erheblich beeinflusst und lernbare Mechanismen statischen Operatoren wie Mean oder Max überlegen sein können. Vinyals et al. demonstrieren mit Set2Set [17] durch algorithmische Experimente, dass eine reihenfolgeunabhängige, iterative Verarbeitung von Mengen die Lern- und Generalisierungsfähigkeit von Modellen signifikant steigert, Lee et al. bauen darauf mit dem Set Transformer [7] auf, der Self-Attention zur Kodierung von Element-Interaktionen

nutzt und dabei die quadratische Komplexität klassischer Attention auf lineare Komplexität reduziert.

Als Grundlage für die in dieser Arbeit verwendeten Attention- und Masking-Mechanismen dienen Vaswani et al. [15], die die Transformer-Architektur einführen, in der Padding und Masking die effiziente Batch-Verarbeitung variabel langer Sequenzen ermöglichen. Veličković et al. übertragen dieses Prinzip mit Graph Attention Networks (GATs) [16] auf graphbasierte Strukturen und nutzen Masked Self-Attention zur Berechnung von Knoten-Repräsentationen ohne kostspielige Matrixoperationen. Liu et al. [8] liefern einen systematischen Überblick über Pooling-Operationen im Deep Learning und bilden damit die kategorische Einordnung der in dieser Arbeit evaluierten Aggregationsmethoden.

2.3 Kollaboratives Threat-Report-Sharing, Gewichtungsstrategien und Datenqualität

Das kollaborative Teilen von Threat-Reports zwischen Knoten erfordert Mechanismen zur Bewertung der Vertrauenswürdigkeit einzelner Knoten. Zhang, Miao & Xue [20] schlagen ein blockchain-basiertes Modell vor, das ein verteiltes Reputationsmanagementsystem integriert, dabei zeigen sie, dass ein Ungleichgewicht zwischen normalen und kompromittierten Knoten die Aggregation systematisch verzerren kann, ein Phänomen, das in der vorliegenden Arbeit als Echo-Chamber-Effekt formalisiert und durch asymptotisches Sättigungsverhalten der Gewichtungsfunktionen strukturell begrenzt wird. Zur Modellierung dieses abnehmenden Grenznutzens greift die Arbeit auf die von Norstad [10] beschriebene Negative Exponential Utility Function mit konstanter absoluter Risikoaversion (CARA) zurück, sowie auf die von Pérez-Maldonado et al. [12] formal hergeleitete logistische Sigmoid-Funktion als S-förmige Alternative.

Hinsichtlich Datenqualität zeigen Hasan et al. [5] in einem systematischen Review, dass fehlende Werte die Performance von ML-Modellen erheblich beeinflussen, eine Grundlage für die implementierte NaN-Fehlerbehandlung in der Threat-Report-Generierung. Die Entscheidung für das CSV- gegenüber dem Parquet-Format wird durch Analysen von Taylor-Davies [14] sowie CellerData [2] gestützt, die belegen, dass der Metadaten-Overhead von Parquet bei kleinen Dateien unter 30 kB die Vorteile der spaltenorientierten Kompression überwiegt.

3 Systemarchitektur und Detektion Framework

In diesem Kapitel wird die zugrundeliegende Systemarchitektur vorgestellt, die sich im Wesentlichen in eine lokale Detektionsdomäne und eine globale P2P-Kommunikationsdomäne gliedert.

3.1 Systemüberblick und verteilte Kommunikation

Das ReSiLENT-System gliedert sich in zwei logische Domänen: die **Charging Station** mit dem eingebetteten Detektion-Client sowie das **Cloud Dashboard** zur zentralen Visualisierung und Auswertung. Beide Domänen sind über ein privates Peer-to-Peer-Netzwerk auf Basis des *Inter Planetary File System* (IPFS) lose gekoppelt. Anstelle direkter IP-zu-IP-Verbindungen erfolgt der Datenaustausch über inhaltsadressierte Bezeichner (*Content Identifier*).

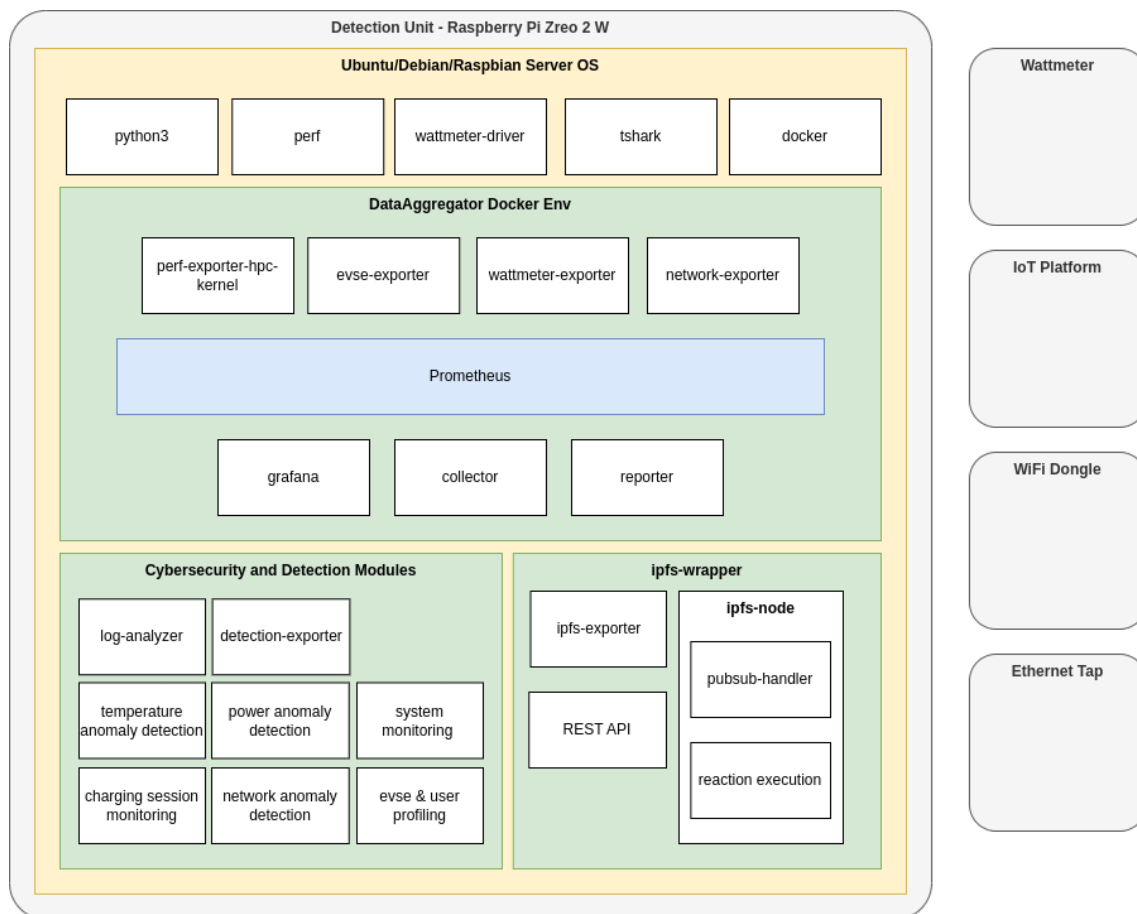


Abbildung 3.1: Gesamtarchitektur des ReSiLENT-Systems mit Charging Station und Cloud Dashboard

Abbildung 3.1 zeigt die Gesamtarchitektur des Systems mit Fokus auf die in jeder Wallbox eingebettete Detektionseinheit. Als zentrales Laufzeitmodul fungiert der *DataAggregator*, der die erfassten Sensordaten der Wallbox über Prometheus bereitstellt und sowohl den Cybersecurity- und Detektions-Modulen als auch Werkzeugen zur Datenaufbereitung zugänglich macht.

Die Architektur ist auf kollaborative Angriffserkennung ausgelegt: Über den *ipfs-wrapper* und dessen **Publish-Subscribe-Mechanismus** (detailliert in Abbildung 3.2 und 3.3) teilen Ladesäulen Kontextinformationen mit allen Netzwerkteilnehmern und könnten somit die situative Wahrnehmung über die gesamte Ladeinfrastruktur hinweg verbessern.

3.1.1 Privates IPFS-Netzwerk und Peer-to-Peer-Datenaustausch

Für die verteilte Kommunikation zwischen den Detektionseinheiten und dem Cloud Dashboard wird ein privates IPFS-Netzwerk eingesetzt. Im Gegensatz zum öffentlichen IPFS-Netz wird der Zugang durch einen gemeinsamen *Swarm Key* beschränkt, sodass ausschließlich autorisierte Knoten dem Netzwerk beitreten können. Die Netzwerktopologie ist in Abbildung 3.2 dargestellt.

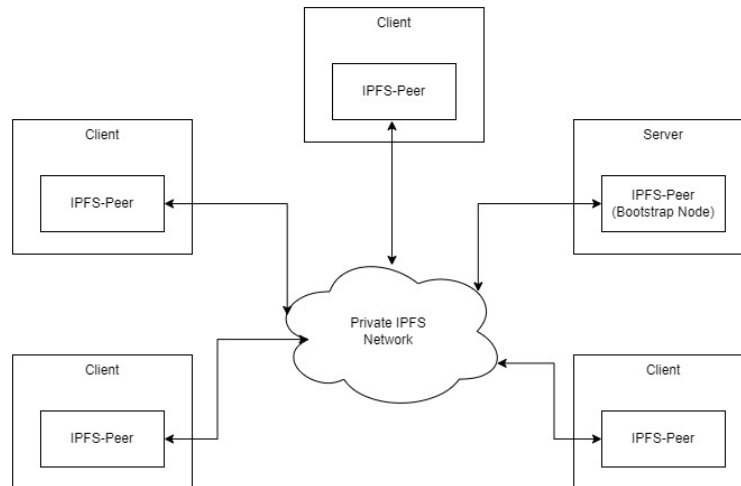


Abbildung 3.2: Topologie des privaten IPFS-Netzwerks mit Bootstrap-Knoten und Client-Peers

Der Netzbeitritt erfolgt über einen dedizierten **Bootstrap-Knoten**, der auf dem Server des Cloud Dashboards betrieben wird. Neue Peers melden sich beim Bootstrap-Knoten an, erhalten die Multiaddresses (*MADDR*) der bekannten Peers und kommunizieren anschließend direkt miteinander, ohne den Bootstrap-Knoten als Relay zu verwenden. Jeder Knoten betreibt einen lokalen **IPFS-Peer-Daemon**, der über den *ipfs-wrapper*, wie in Abbildung 3.3 dargestellt, in die Systemarchitektur eingebunden ist.

Für die Benachrichtigung über neue Daten wird der **Publish-Subscribe-Mechanismus** von IPFS genutzt: Sobald neue Messdaten erhoben werden, können diese veröffentlicht werden indem die Wallbox die zugehörige CID zusammen mit der eigenen MADDR über einen gemeinsam abonnierten Kanal an alle Teilnehmer versendet. Die Empfänger rufen die Datei daraufhin direkt beim Ursprungs-Peer ab. Da eine CID dem kryptografischen Hash des jeweiligen Inhalts entspricht, ist die Datenintegrität implizit gewährleistet, ohne dass zusätzliche Signaturmechanismen erforderlich sind.

3.1.2 End-to-End-Datenfluss

Der vollständige Datenfluss vom Sensor bis zur Visualisierung lässt sich in drei Phasen unterteilen: Datenerhebung und Aggregation auf der Charging Station, Transport über das IPFS-Netz sowie Persistierung und Darstellung im Cloud Dashboard. Das Sequenzdiagramm in Abbildung 3.4 sowie der Datenfluss in Abbildung 3.3 illustrieren diesen Ablauf.

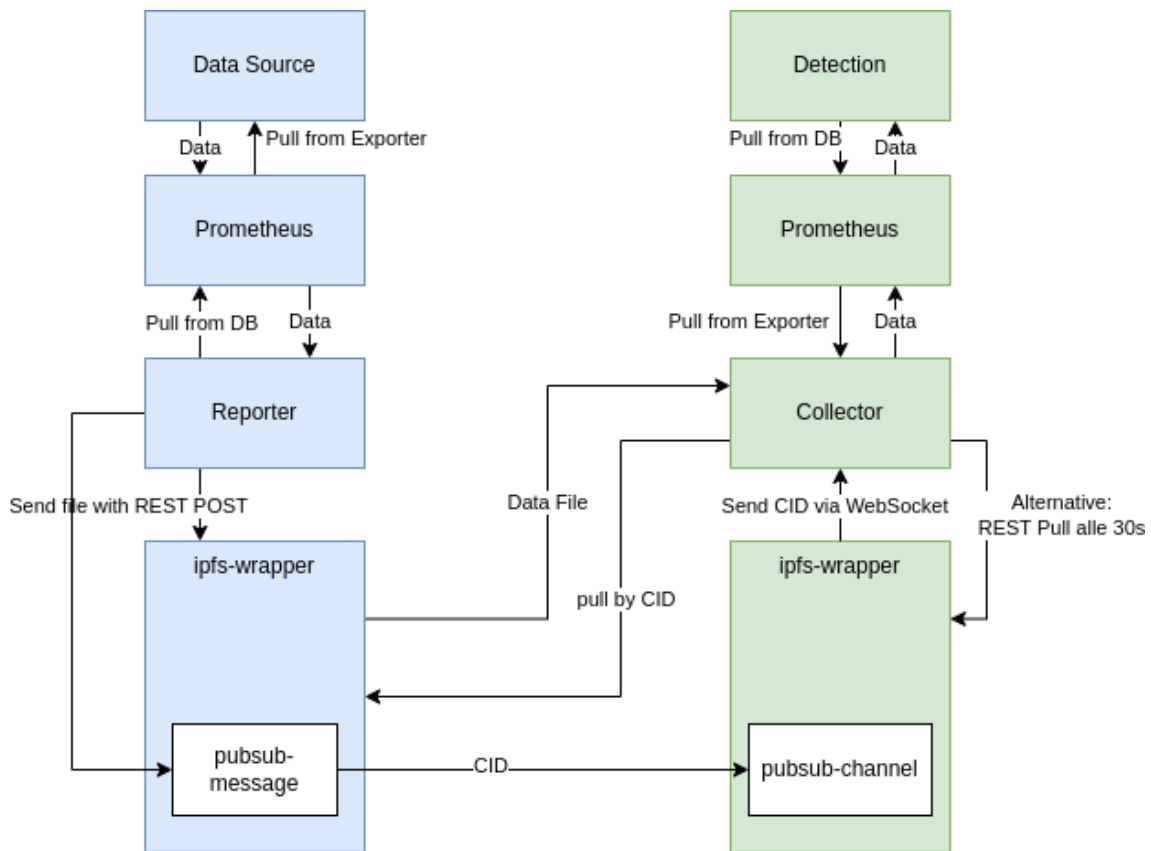


Abbildung 3.3: Datenfluss zwischen zwei Charging Stationen über IPFS

Datenerhebung (Charging Station)

Der *Aggregator* innerhalb des Knotens sammelt die Messdaten. Prometheus scraped alle Exporter in konfigurierbaren Intervallen. Der *Reporter* liest die aggregierten Daten aus der Prometheus-Datenbank und übermittelt sie an den *ipfs-wrapper*.

Transport (IPFS-Netzwerk)

Der *ipfs-wrapper* nimmt die Datei entgegen, publiziert sie im lokalen IPFS-Knoten und erhält eine CID zurück. Anschließend wird die CID zusammen mit der eigenen MADDR über den konfigurierten **Publish-Subscribe-Mechanismus** im privaten IPFS-Netz verbreitet. Die eigentlichen Messdaten werden dabei nicht aktiv gepusht, lediglich der Zeiger (CID) wird verteilt.

Persistierung und Visualisierung (Cloud Dashboard)

Auf der Serverseite empfängt der *Publish-Subscribe-Event-Listener* des *ipfs-wrapper* die CID und MADDR und leitet diese per **WebSocket** an den Collector weiter. Der Collector fordert daraufhin die vollständige Datei über den lokalen IPFS-Peer an, der sie in Chunks vom Ursprungs-Peer abrufen. Der *File Extractor* verarbeitet die Rohdaten und persistiert die Messwerte in **InfluxDB**. Grafana greift auf InfluxDB zu und stellt die Zeitreihendaten im Dashboard dar (siehe Abbildung 3.5).

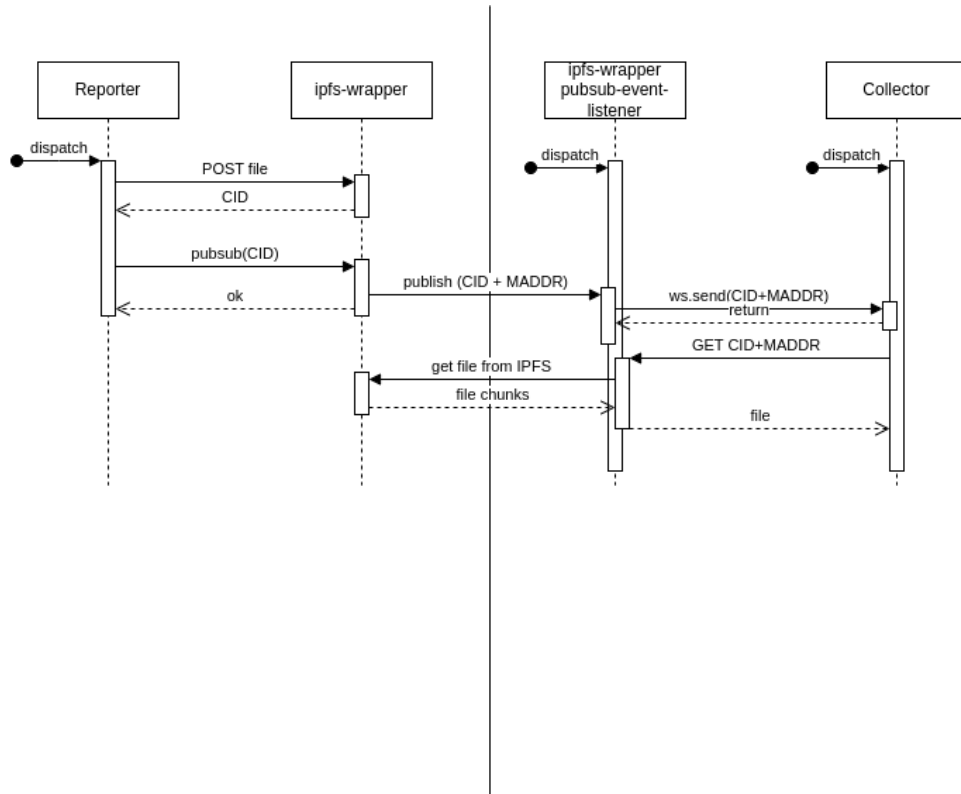


Abbildung 3.4: Sequenzdiagramm des IPFS-basierten Datenaustausches

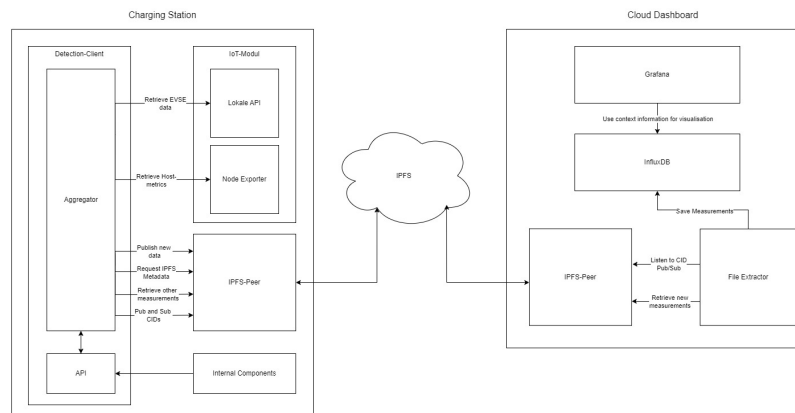


Abbildung 3.5: Komponenten des Cloud Dashboards: IPFS-Peer, File Extractor, InfluxDB und Grafana

Dieses Verständnis der Systemarchitektur und des Datenflusses bildet die Grundlage für die folgende Arbeit zur Implementierung des Threat-Reports und dessen spätere Aggregation.

3.2 Physische Testumgebung

Die physische Testumgebung für die Implementierung und Evaluierung der vorliegenden Arbeit besteht aus einer realen ChargeIQ-Wallbox. Die Detektion-Module sind in diesem Szenario abweichend von der in Kapitel 3.1 (Systemüberblick und verteilte Kommunikation) beschriebenen

Zielarchitektur, nicht auf einem Raspberry Pi Zero 2 W, sondern zu Testzwecken auf einem leistungsfähigeren Raspberry Pi 4 B und BananaPi M5 installiert. Diese sind über eine kabelgebundene Ethernet-Verbindung in das private IPFS-Netzwerk eingebunden, um eine stabile und performante Kommunikation zu gewährleisten. Die Wallbox selbst ist über eine drahtlose Verbindung in dasselbe private IPFS-Netzwerk integriert.

Darüber hinaus besteht eine serielle *I2C*-Verbindung (*Inter-Integrated Circuit*) zwischen der ChargeIQ-Wallbox und einem Einplatinencomputer in diesem Fall, der Raspberry Pi 4 B, welcher stellvertretend für die spätere Detektionseinheit auf Basis des Raspberry Pi Zero 2 W fungiert. Diese Verbindung ermöglicht die Erfassung von Echtzeitdaten des integrierten Wattmeters, welcher die Sensordaten für eine im Kapitel 3.3 (Detektionseinheiten der Ladeinfrastruktur) beschriebene Einheit bereitstellt.

Zusätzliche Sensordaten werden dem Raspberry Pi 4 B über die in der Testumgebung eingerichtete drahtlose WLAN-Verbindung übermittelt, in welche sowohl die als Detektionseinheit eingesetzte Einplatinencomputer als auch die Wallboxen eingebunden sind.

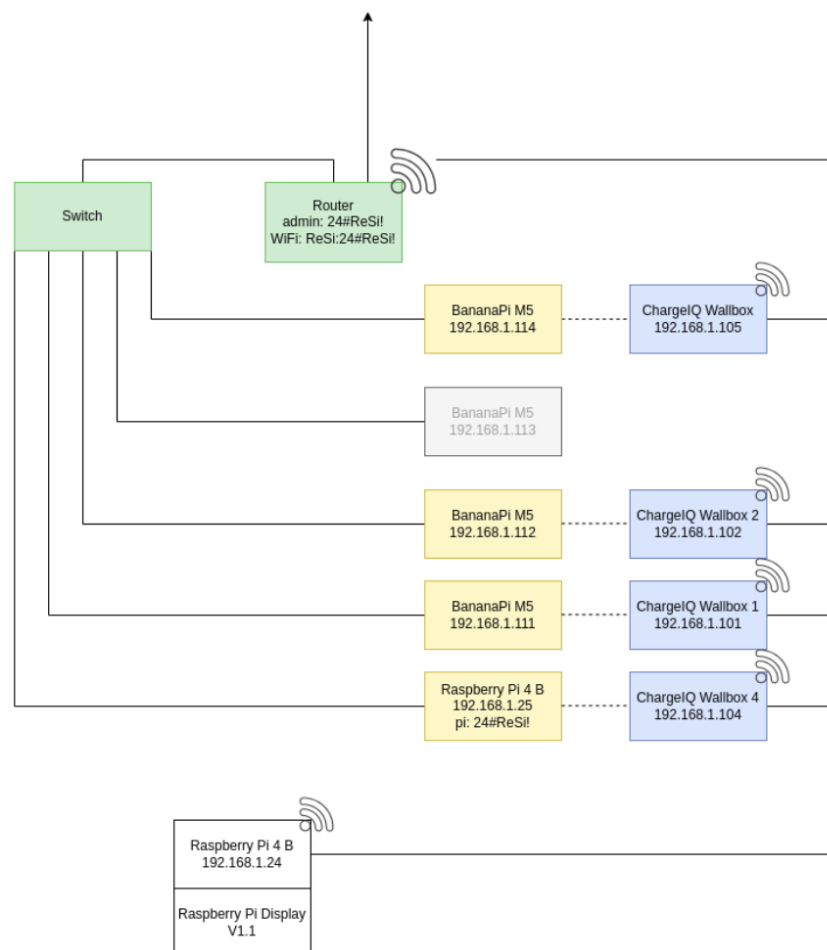


Abbildung 3.6: Schematische Darstellung der physischen Testumgebung.

3.3 Detektionseinheiten der Ladeinfrastruktur

Im Folgenden werden die vier Hauptdetektionseinheiten der Testumgebung vorgestellt, welche für den *Threat-Report* sowie den *Aggregierten-Threat-Report* von zentraler Bedeutung sind.

3.3.1 System- und Leistungsanomaliedetektion

Die Einheit *pi_anomaly_detection* trifft Entscheidungen anhand der Metriken Shunt-Spannung, Bus-Spannung, Stromstärke und Leistung. Diese Messdaten werden von den Sensoren der Wall-box erfasst und an Prometheus weitergeleitet. Die beschriebene Detektionseinheit greift diese Daten über Prometheus ab und nutzt zwei Deep-Learning-Modelle:

- **xLSTM-Autoencoder:** Dieses Modell operiert auf einem Sliding-Window von $w = 44$ Sekunden und lernt die Normalverteilung der physikalischen Parameter (Spannung, Strom, Leistung). Eine Anomalie wird detektiert, wenn der Rekonstruktionsfehler E_t den Schwellenwert $\tau = 0,008$ überschreitet. Die Wahrscheinlichkeit wird über eine Sigmoid-Funktion approximiert:

$$P_{xlstm} = \frac{1}{1 + e^{-50 \cdot (E_t - \tau)}}$$

- **Transformer-Klassifikator:** Parallel dazu extrahiert das System mittels *tsfresh* 15 statistische Merkmale aus einem 60-Sekunden-Fenster. Ein Transformer-Modell klassifiziert diese aggregierten Merkmale direkt in die Kategorien *benign* oder *attack*.
- **Sequentielles Voting-Verfahren:** Zur Stabilisierung der Detektion nutzt das System einen Deques-Puffer der Länge $k = 3$. Ein Alarm wird nur ausgelöst, wenn:
 - Beide Modelle im aktuellsten Zeitschritt eine Anomalie melden ODER
 - Mindestens eines der Modelle eine konsistente Sequenz von drei aufeinanderfolgenden positiven Detektionen aufweist.

Die Detektionseinheit reagiert insbesondere auf Stromspitzen und -einbrüche.

3.3.2 Physikalische Signal- und Temperaturüberwachung

Die Detektionseinheit *detection_power_temperature* arbeitet mit einer klassischen regelbasierten Logik, welche IDS-Muster zuordnet und auf Basis von Schwellenwerten für physikalische Parameter (Spannung, Leistung, Temperatur) Anomalien detektiert. Die Detektion ist in zwei Bereiche unterteilt:

- **Temperaturüberwachung:** Diese Anomalieerkennung ruft die Temperaturdaten der letzten 60 Sekunden von Prometheus ab und erkennt Anomalien anhand von drei definierten Regeln:
 - *Absolute Obergrenze:* Wenn die Temperatur im abgefragten Zeitfenster einen Wert von 70°C überschreitet, wird eine Anomalie detektiert.
 - *Konstantreihe + Sprung:* Wenn ein Temperatursprung bei einer ansonsten konstanten Reihe eine Toleranz von > 1 überschreitet, wird eine Anomalie gemeldet.
 - *Anomalie durch Z-Score:* Wenn die Standardabweichung der Temperaturwerte einen Z-Score von $> 3,0$ erreicht, löst die Anomalieerkennung aus.
- **Wattmeter-Überwachung:** Analog zur System- und Leistungsanomaliedetektion (siehe Kapitel 3.3.1 (System und Leistungsanomaliedetektion)) ruft diese Detektion ebenfalls vier Metriken zu Spannung und Leistung der letzten 60 Sekunden ab. Die Detektion prüft, ob

diese Werte fest definierte Schwellenwerte (Minimum oder Maximum) unter- bzw. überschreiten.

Sobald die Temperatur- und/oder die Wattmeter-Überwachung eine positive Anomalie detektieren, schlägt auch der kombinierte Alarm *detection_power_temperature* aus.

3.3.3 KI-basierte Angriffserkennung im Netzwerk

Die Detektionskomponente *ids_power_ai_detection* analysiert dieselben vier physikalischen Messgrößen, die in Kapitel 3.3.1 (System- und Leistungsanomaliedetektion) beschrieben wurden, verzichtet jedoch vollständig auf statische Schwellenwerte. Stattdessen kommen zwei komplementäre Machine-Learning-Modelle zum Einsatz, die anomales Verhalten im zeitlichen Kontext erkennen:

- *Isolation Forest*: Dies ist ein Verfahren, welches einzelne Messpunkte anhand ihrer statistischen Isolation im Merkmalsraum bewertet. Für jeden Messzeitpunkt wird ein Anomalie-Score berechnet. Weicht ein Datenpunkt signifikant vom gelernten Normalzustand ab, wird dies als Anomalie gewertet.
- *LSTM-Autoencoder*: Wie für LSTM-Modelle (Long Short-Term Memory) üblich, betrachtet dieses Modell nicht nur einzelne Messwerte, sondern eine Sequenz von 60 Sekunden (bei einem Abfrageintervall von 5 Sekunden). Das Modell wurde auf normalem Betriebsverhalten trainiert und lernt, typische Zeitreihenmuster zu rekonstruieren. Zur Erkennung wird der *Mean Squared Error* (MSE) zwischen der originalen Sequenz und ihrer Rekonstruktion berechnet. Überschreitet der MSE des aktuellen Fensters einen empirisch bestimmten Schwellenwert ($LSTM_THRESHOLD = 0,01$), wird eine Anomalie ausgelöst.

3.3.4 Hardware-Ereignisüberwachung

In dem Verfahren der *detection_hardwareevents* werden knapp 20 relevante Hardware-Ereignisse kontinuierlich bewertet und mittels eines LSTM-Modells auf anomales Verhalten geprüft.

Das Modell analysiert die Daten in einem gleitenden Zeitfenster. Ergibt die Modellvorhersage eine Angriffswahrscheinlichkeit von über 50 %, wird das Ereignis als Anomalie klassifiziert. Die Ergebnisse, einschließlich der Einzelwahrscheinlichkeiten für *benign* und *attack*, werden über eine dedizierte Schnittstelle an Prometheus exportiert und zur Verfügung gestellt.

4 Konzeption des Threat Reporters

In diesem Kapitel wird das konzeptionelle Fundament des Threat-Reporters erarbeitet. Um eine netzwerkweite Anomalieerkennung zu ermöglichen, müssen die lokal generierten Daten zunächst einheitlich aufbereitet werden. Die folgenden Abschnitte beschreiben dazu das zentrale Datenmanagement, die zeitliche Diskretisierung der Messfenster sowie die statistische Aggregation der Merkmale.

4.1 Zentralisiertes Datenmanagement und Merkmalsextraktion für den Threat-Report

Um die Kompatibilität und Standardisierung des Threat-Reports zu gewährleisten, wurde entschieden, die erforderlichen Daten zentral aus der Lokalen Prometheus-Datenbank zu extrahieren, anstatt diese einzeln von den jeweiligen Detektionseinheiten abzurufen. Hierfür wurden spezifische Prometheus-Exporter für die Detektionseinheiten entworfen und integriert. Diese stellen die Daten kontinuierlich über die lokalen Ports 2112, 2113, 2115 und 2116 bereit.

Die über die Exporter gelieferten Datensätze umfassen sowohl die Klassifizierung der Anomalien (Attack oder Benign) als auch sämtliche Features, die für die jeweilige Detektion herangezogen wurden. Dieser umfassende Ansatz ist notwendig, da eine zukünftige Optimierung der Gesamtdetektion mittels eines ML-Modells vorgesehen ist. Um ein qualitativ hochwertiges Training des Modells zu ermöglichen, kann eine möglichst breite Basis an Inputdaten den Lernprozess signifikant verbessern [6].

Die Grundlage hierfür bildete eine detaillierte Analyse der Detektionseinheiten sowie ein tiefgreifendes Verständnis der bereitgestellten Datenstrukturen, wie bereits in Kapitel 3.3 (Detektionseinheiten der Ladeinfrastruktur) dargelegt.

In 10.1 (`reporter.py`) werden diese Daten über Prometheus importiert und in ein CSV-Format überführt. Das CSV-Format wurde aufgrund seiner weitreichenden Kompatibilität und einfachen Handhabung und Menschenlesbarkeit gewählt. Zusätzlich überschreiten die generierten Threat-Reports eine Dateigröße von maximal 30 kB nicht und somit bieten zeilenbasierte Formate wie CSV oder JSON gegenüber spaltenorientierten Formaten wie *Parquet* keine Nachteile. Im Gegenteil: Der Parquet-spezifischer Metadaten-Overhead kann bei derart kleinen Dateien den Großteil der Dateigröße ausmachen, während die Vorteile (Kompression, spaltenbasierter Zugriff) erst bei Dateigrößen im Bereich mehrerer hundert Megabyte relevant werden [2, 14].

4.2 Zeitliche Diskretisierung und Intervalldefinition

Um die Systemressourcen des *Raspberry Pi Zero 2 W* zu schonen und gleichzeitig das Datenaufkommen zu begrenzen, ohne die Detektionssicherheit zu beeinträchtigen, wurde ein Zeitfenster von jeweils 60 Sekunden gewählt, das in 30-Sekunden-Intervallen verschoben ist. Dieser Ansatz ermöglicht es, umfassende Informationen der Detektionseinheiten zu erhalten. Die Parameter für diese Zeitintervalle wurden bewusst variabel gestaltet (siehe Listing 10.1 (`reporter.py`)). Dies

schafft die Grundlage für zukünftige Untersuchungen, wie sie in Kapitel 8 (Diskussion und Forschungslücke) beschrieben sind, um den Einfluss unterschiedlicher Zeitfenster auf die Qualität der Threat-Reports empirisch zu evaluieren.

Die aktuell implementierte Sequenz umfasst drei Datenpunkte: den aktuellen Zeitpunkt (0 Sekunden) sowie die Zustände der Zeitfenster von vor 30 und 60 Sekunden. Durch diese Struktur wird eine Timeline generiert, die den zeitlichen Verlauf der Detektion über drei Messfenster abbildet, welches eine verbesserte Darstellung des zeitlichen Verlaufs ermöglicht, anstatt isolierte Zeitpunkte zu betrachten [6, 19].

4.3 Statistische Merkmalsaggregation mittels Sliding-Window-Analyse

Neben den aktuellen Merkmalswerten (Features) und Anomalie-Indikatoren der Detektionseinheit werden zusätzlich historische Daten mittels *Sliding Windows* importiert. Pro Threat-Report werden drei dieser Fenster definiert, welche jeweils den aktuellen Datenpunkt in Relation zu zwei vorangegangenen Zeitpunkten setzen:

- **Window 1:** 0 Sekunden bis -60 Sekunden
- **Window 2:** -30 Sekunden bis -90 Sekunden
- **Window 3:** -60 Sekunden bis -120 Sekunden

Innerhalb dieser Zeitfenster werden für die statistische Auswertung anhand klassischer Kennzahlen berechnet [6] [1]:

Die statistische Auswertung der Merkmale im Zeitverlauf umfasst folgende Kennzahlen:

- **Mittelwert:** *Average* der Durchschnittswert innerhalb des Fensters.
- **Extremwerte:** *Minimum* und *Maximum* im jeweiligen Intervall.
- **Median:** Der Zentralwert der Datenpunkte.
- **Anstieg (Increase):** Die relative oder absolute Veränderung innerhalb des Fensters.
- **Summe:** Die kumulierten Werte des jeweiligen Zeitraums.

Siehe Abbildung 7.3

Durch diese Methodik entsteht ein umfassender Threat-Report, der nicht nur eine Momentaufnahme der Detektionseinheiten liefert, sondern auch die zeitliche Entwicklung über die letzten 60 Sekunden bzw. durch die Sliding Windows bis zu 120 Sekunden in die Vergangenheit abbildet. Diese Datenbasis dient als Input für eine spätere Aufbereitung und Training eines ML-Modells, die in Kapitel 8 (Diskussion und Forschungslücke) beschrieben sind.

Der für diesen Abschnitt implementierte Quellcode, welcher die Logik der Datenabfrage sowie die Generierung des Threat-Reports umfasst, ist im Anhang unter Listing 10.1 (`reporter.py`) und Listing 10.2 (`queries.py`) dokumentiert.

5 Theoretische Grundlagen der Informationsfusion

Dieses Kapitel widmet sich den mathematischen und algorithmischen Grundlagen der Informationsfusion bei variablen Eingangsgrößen.

5.1 Vergleich von Aggregationsmethoden

Nun, da jeder Knoten automatisiert einen umfangreichen Threat-Report erstellt und eine Vielzahl von Informationen über die Bedrohungslage an seine Nachbarn weitergeben kann, stellt sich die Frage, wie diese Informationen am effektivsten zur verbesserten Erkennung von Angriffen des einzelnen Knotens genutzt werden können. Hierbei ist folgende Problemstellung umfassend zu betrachten:

- Jeder Knoten erstellt einen standardisierten Threat-Report, wie in Kapitel 4 (Konzeption des Threat-Reporters) beschrieben, welcher detaillierte Informationen über die lokale Bedrohungslage enthält.
- Die Informationen werden an die Nachbarn in einem Peer-to-Peer-Netzwerk über die Pub/Sub-Infrastruktur weitergegeben, wie in Kapitel 3 (Systemarchitektur und Detektion Framework) dargelegt. Dies erfolgt in einem festen Intervall von 60 Sekunden.
- Für die verbesserte Erkennung stehen jedem einzelnen Knoten zu jedem Intervall eine variable Anzahl an Threat-Reports von den Nachbar Knoten zur Verfügung. Dies ist dem dynamischen Aufbau der Netzwerktopologie sowie potenziellen Latenzen geschuldet.
- Es gilt, eine Methode zu entwickeln, welche die Relevanz der einzelnen Knoten bewertet und somit die Gewichtung der Informationen jedes einzelnen Threat-Reports dynamisch und intelligent anpasst.

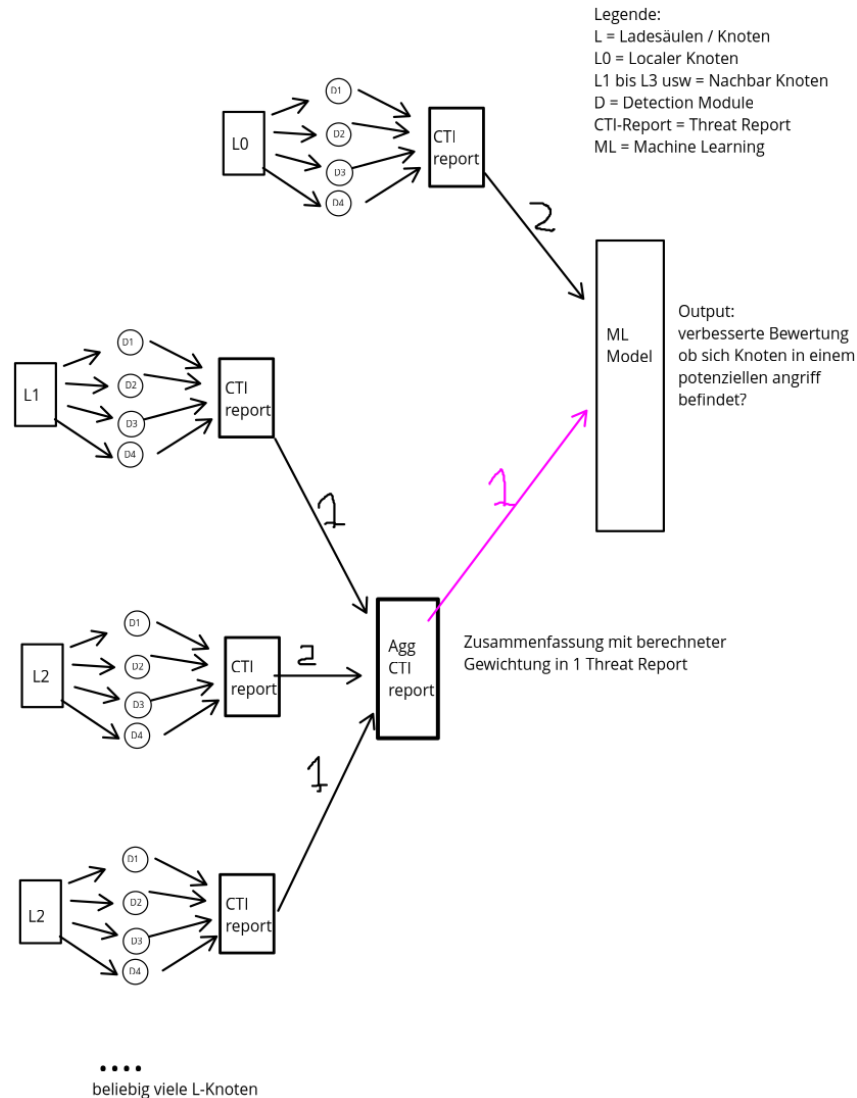


Abbildung 5.1: Aufbau der Fusion von Threat-Reports aus Nachbar Knoten

Eine zentrale Herausforderung bei der Verwendung von ML-Verfahren, insbesondere bei klassischen neuronalen Netzen, besteht darin, dass diese in der Regel eine Eingabe mit fester Dimension (Fixed-Size Input) benötigen [18]. Da die Anzahl der empfangenen Threat-Reports jedoch pro Zeitintervall schwankt, kann keine statische Eingabeschicht definiert werden. Im Folgenden werden daher verschiedene Methoden untersucht, die in der Lage sind, mit dieser Variabilität umzugehen und eine konsistente Datenbasis für das Training und die Inferenz eines ML-Modells sicherzustellen.

5.1.1 Statische Aggregation

Bei der statischen Aggregation liegt der Fokus auf vorab manuell definierten Regeln. Diese sind regelbasiert konzipiert und verfügen über keine lernenden Komponenten. Ein typisches Szenario für diesen Ansatz ist die situationsbezogene Aggregation: Hierbei werden unterschiedliche Alerts nur dann zu einer Situation zusammengefasst, wenn sie dieselbe Quelle, dasselbe Ziel und dieselbe Angriffsklasse aufweisen. Der Detaillierungsgrad dieser Zusammenfassung wird dabei

durch statisch konfigurierte Schwellenwerte gesteuert [3]. Als gängige mathematische Aggregationsmethoden kommen in diesem Kontext nach der Gruppierung die Berechnung von Minimum, Maximum, Durchschnitt oder der Standardabweichung zum Einsatz [7, 8].

5.1.2 Padding und Masking

Padding und Masking sind essenzielle Verfahren bei der Verarbeitung von Sequenzdaten, um die Inferenz mit variierenden Sequenzlängen zu ermöglichen. Diese Mechanismen sind standardmäßig in modernen Deep-Learning-Frameworks integriert, wie beispielsweise im Parameter `key_padding_mask` der `torch.nn.MultiheadAttention`-Klasse in PyTorch [11].

Der Prozess des Paddings definiert eine maximale Sequenzlänge $n_{\max}$. Sequenzen, die diese Länge unterschreiten, werden mit speziellen [PAD]-Token aufgefüllt, bis sie die Dimension $n_{\max}$ erreichen. Damit diese künstlich hinzugefügten Token das Training oder die Aggregation nicht verfälschen, wird eine Maskierung (Masking) eingesetzt. Diese stellt sicher, dass die entsprechenden Positionen während der Berechnung der Attention-Scores ignoriert werden.

In der Arbeit von Vaswani et al. (2017) [15] bildet dieses Prinzip die Grundlage für die effiziente Batch-Verarbeitung innerhalb der Transformer-Architektur. Hierbei verhindert das Masking, dass das Modell Aufmerksamkeit auf Padding-Bereiche richtet (siehe Abbildung 5.2).

Attention Visualizations

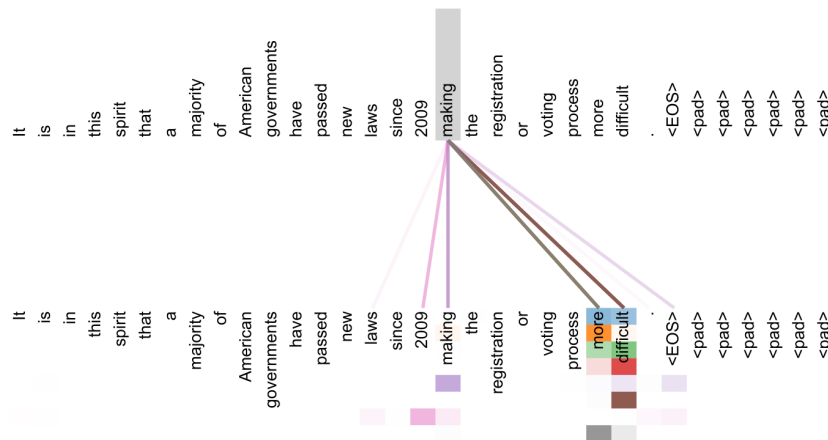


Abbildung 5.2: Visualisierung der Attention-Mechanismen (adaptiert nach Abbildung 3 aus [15])

Ebenso findet dieses Verfahren Anwendung in der BERT-Architektur von Devlin et al. [4]. Hierbei werden Eingabesequenzen auf eine feste Länge von meist 512 Token gepaddet, wobei das Masking sowohl für das Padding als auch für die spezifische Trainingsmethode des *Masked Language Modeling* (MLM) verwendet wird.

5.1.3 Deep Sets

Im Gegensatz zu den in Kapitel 5.1.2 (Padding und Masking) beschriebenen Verfahren, die auf einer festen Anzahl von Elementen basieren, verfolgen *Deep Sets* einen mengenbasierten Ansatz. Dabei wird die Eingabe als eine ungeordnete Menge X betrachtet. Das Modell lernt hierbei, die

Relevanz der einzelnen Elemente autonom zu bewerten und die Gewichtung der Informationen dynamisch anzupassen.

Eine fundamentale Anforderung an die verarbeitende Funktion $f(X)$ ist die *Permutationsinvarianz*. Dies bedeutet, dass die Ausgabe der Funktion unabhängig von der Reihenfolge der Elemente innerhalb der Menge sein muss [18].

Theorem 1 (Zaheer et al., 2017 [18]). *Eine Funktion $f(X)$ auf einer Menge X , deren Elemente aus einem abzählbaren Universum stammen, ist genau dann permutationsinvariant, wenn sie sich wie folgt dekomponieren lässt:*

$$f(X) = \rho \left(\sum_{x \in X} \phi(x) \right)$$

wobei ϕ und ρ geeignete Transformationen (typischerweise neuronale Netze) darstellen.

Während die Summenbildung die theoretische Grundlage für die Aggregation bildet, werden in der Praxis häufig auch der Mittelwert (*Mean*) oder das Maximum (*Max*) verwendet, da diese Operationen ebenfalls die Bedingung der Permutationsinvarianz erfüllen. Soelch et al. [13] zeigen jedoch auf, dass die Wahl der Aggregationsfunktion einen erheblichen Einfluss auf die Modellleistung hat, und schlagen als Alternative lernbare Aggregationsmechanismen vor.

5.1.4 Set2Set

Obwohl auch die vorherigen Aggregationsmethoden variable Eingabelängen verarbeiten können, unterscheiden sie sich von Set2Set primär in der Art der Informationsverdichtung. Während herkömmliche Methoden die Eingabemenge meist durch eine statische Aggregationsfunktion (z. B. Mean-Pooling) in einen festen Merkmalsraum transformieren, nutzt Set2Set einen iterativen *Read-Process*. Durch die Kombination aus Attention-Mechanismen und einem wiederkehrenden Zustand kann das Modell die Menge mehrfach durchlaufen. Dies ermöglicht es, komplexe Abhängigkeiten zwischen den Elementen aufzulösen, die bei einer einfachen Summation oder Mittelwertbildung verloren gingen [17].

Algorithmische Experimente, wie das Sortieren von Zahlenfolgen, belegen, dass Modelle eine höhere Lern- und Generalisierungsfähigkeit aufweisen, wenn sie nicht durch eine zufälligen Reihenfolge der Eingabedaten eingeschränkt werden [17]. Auch die in Kapitel 5.1.2 (Padding und Masking) behandelten Transformer-Architekturen werden mit einer Form des Set2Set-Ansatzes erweitert [15].

5.1.5 Graph Neural Networks und Pooling

Graph Neural Networks (GNNs) bieten leistungsstarke Klassen von Modellen zur Verarbeitung von Daten, die in Form von Graphen vorliegen. Knoten repräsentieren dabei Entitäten (z. B. IP-Adressen), während Kanten die Beziehungen zwischen diesen Entitäten modellieren und die zugehörigen Flow-Informationen als Kantenmerkmale tragen [9].

Durch iteratives Aggregieren der Nachbarschaftsinformationen können GNNs sowohl topologische Muster als auch merkmalsbasierte Strukturen im Netzwerkgraphen erkennen. Das Pooling erfolgt hier, mittels Mean-Pooling über die Kantenmerkmale der Nachbarschaft [9].

6 Methodik: Gewichtete Aggregation von Threat Reports

Für eine verlässliche Lagebeurteilung müssen die eingehenden Sicherheitsberichte je nach Relevanz und Informationsdichte dynamisch gewichtet werden. In diesem Abschnitt werden die mathematischen Anforderungen an die Gewichtung definiert und zwei konkrete Funktionsansätze vorgestellt.

6.1 Anforderungen an die Gewichtungsfunktion

Für die Konzeption der Gewichtungsfunktion ergeben sich aus dem Anwendungskontext der Threat-Reports folgende zentrale Anforderungen:

- **Erzeugung fester Merkmalsvektoren:** Da die Anzahl der liefernden Knoten und damit der eingehenden Threat-Reports im operativen Betrieb kontinuierlich variiert, muss die Gewichtungsfunktion eine beliebige Anzahl von Eingangsmengen verarbeiten können. Ziel ist die Extraktion einer gleichbleibenden Informationsmenge (Fixed-Size Input) aus diesen variablen Eingangsdaten. Dies stellt sicher, dass der resultierende Merkmalsvektor unabhängig von der ursprünglichen Report-Menge eine einheitliche Struktur aufweist, was die notwendige Voraussetzung für die Performanz und Stabilität nachgelagerter ML-Algorithmen ist [16, 18].
- **Relevanzbasierte Gewichtung der Detektionen:** Um die Qualität der aggregierten Ergebnisse zu steigern, sollte das Modell die Relevanz der einzelnen Threat-Reports differenziert auf Basis der enthaltenen Einzeldetektionen bewerten. Diese Gewichtungsstrategie orientiert sich an modernen Attention-Mechanismen, wie sie in Kapitel 5.1 (Vergleich von Aggregationsmethoden) detailliert evaluiert wurden. Die Notwendigkeit einer solchen Attention-Mechanismusses ergibt sich aus der Anforderung, die Informationsdichte einzelner Quellen präzise abzubilden und signifikante Threat-Reports gegenüber weniger aussagekräftigen Threat-Reports zu priorisieren.
- **Asymptotisches Sättigungsverhalten der Gewichtung:** Um eine übermäßige Dominanz einzelner Threat-Reports mit extrem hohen Detektionszahlen zu vermeiden, darf die Gewichtungsfunktion kein unbegrenztes Wachstum aufweisen. Gefordert wird ein fester Grenzwert (Asymptote), damit zusätzliche Detektionen ab einem gewissen Schwellenwert nur noch marginalen Einfluss auf das Gesamtgewicht haben. Dieser Mechanismus soll den Echo-Chamber-Effekt reduzieren (siehe Kapitel 7.3 (Analyse des Echo-Chamber-Effekts)) und verhindern, dass das Aggregationsergebnis durch redundante Meldungsmengen verzerrt wird.

6.2 Auswahl der Gewichtungsfunktion

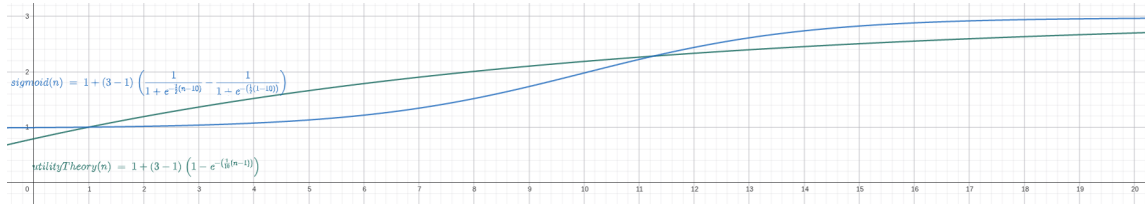


Abbildung 6.1: Vergleich der Sigmoid-Funktion und der Utility-Theory-Funktion. Die X-Achse zeigt die Anzahl positiver Detektionen abzüglich des Initialwertes ($n - 1$), die Y-Achse den resultierenden Gewichtungsfaktor.

6.2.1 Utility-Theory-Funktion

Asymptotische Gewichtungsfunktion: Zur Modellierung der Relevanz wird eine Funktion gewählt, die dem Prinzip des abnehmenden Grenznutzens aus der Entscheidungstheorie folgt. Hierzu wird die *Negative Exponential Utility Function* herangezogen, welche durch eine konstante absolute Risikoaversion (CARA) charakterisiert ist [10, Sec. 8].

Durch eine positive affine Transformation [10, Sec. 3] wird die Funktion so skaliert, dass sie monoton steigend gegen einen anwendungsspezifisch definierten Grenzwert $W_{\max}$ konvergiert, die konkrete Wahl dieses Grenzwerts sowie die Skalierung sind nicht Teil der zitierten Theorie, sondern eigene Entwurfsentscheidungen. Zusätzliche Detektionen stellen dabei einen abnehmenden Grenzinformationsgewinn (*diminishing returns*) dar, entsprechend dem Prinzip des abnehmenden Grenznutzens [10, Sec. 1]. Dies verhindert eine Verzerrung durch Threat-Reports mit extrem vielen positiven Einzeldetektionen und stellt eine robuste Aggregation sicher.

Die Gewichtung der Threat-Reports erfolgt über die folgende Funktion:

$$w(n) = W_{\text{base}} + (W_{\text{max}} - W_{\text{base}}) \cdot (1 - e^{-K \cdot (n-1)}) \quad (6.1)$$

Wie die Kurve (Abbildung 6.1) verdeutlicht, erfahren Threat-Reports mit einer steigenden Anzahl positiver Detektionen (n) eine kontinuierlich höhere Gewichtung (w). Diese konvergiert gegen einen definierten Grenzwert von $W_{\max} = 3$, wodurch eine übermäßige Dominanz einzelner Threat-Reports mit extrem hohen positiven Einzeldetektionen (Ausreißer) unterdrückt wird. Gleichzeitig stellt der Achsenabschnitt bei $n = 1$ sicher, dass jeder Threat-Report mit einem Basisgewicht von $W_{\text{base}} = 1$ in die Aggregation einfließt, um eine Mindestrelevanz aller vorliegenden Informationen zu gewährleisten.

Die praktische Umsetzung der beschriebenen Gewichtungslogik ist in Listing 10.3 (`aggregation.py`) innerhalb der Funktion `add_report_weight` dokumentiert.

6.2.2 Sigmoid-Funktion

Asymptotische Gewichtungsfunktion mit S-förmigem Verlauf: Als Alternative zur Utility-Theory-Funktion wurde die *Logistische Sigmoid-Funktion* evaluiert. Sie ist im Bereich des maschinellen Lernens weit verbreitet und findet insbesondere in neuronalen Netzen Anwendung,

etwa als Aktivierungsfunktion bei der Klassifikation sowie in Graph Attention Networks zur Aggregation von Nachbarknoten [16].

Durch eine positive affine Transformation sowie eine Verschiebung um den Ankerpunkt wird die Funktion so skaliert und normiert, dass bei $n = 1$ stets $w(1) = W_{\text{base}}$ gilt. Dies stellt sicher, dass jeder Threat-Report mindestens mit dem Faktor W_{base} in die gewichtete Aggregation einfließt. Im Code wird $n = 1$ als Standardwert verwendet, sodass Threat-Reports ohne positive Detektionen ($n_{\text{attacks}} = 0$) mit der Gewichtung $w(1) = W_{\text{base}}$ bewertet werden.

Die Gewichtung der Threat-Reports erfolgt über die folgende Funktion:

$$w(n) = W_{\text{base}} + (W_{\text{max}} - W_{\text{base}}) \cdot \left(\frac{1}{1 + e^{-k(n-n_0)}} - \frac{1}{1 + e^{-k(1-n_0)}} \right) \quad (6.2)$$

Der Subtraktionsterm $\frac{1}{1 + e^{-k(1-n_0)}}$ stellt den Ankerpunkt bei $n = 1$ sicher. Er verschiebt die gesamte Kurve vertikal so, dass $w(1) = W_{\text{base}}$ exakt erfüllt ist, unabhängig von der Wahl der Parameter k und n_0 .

Im Gegensatz zur Utility-Theory-Funktion, die streng konkav verläuft und den stärksten Anstieg bei $n = 1$ aufweist, besitzt die Sigmoid-Funktion durch den Wendepunkt bei n_0 einen S-förmigen Verlauf [12]. Dies modelliert die Annahme, dass eine Mindestzahl an Detektionen überschritten werden muss, bevor ein Threat-Report als signifikant eingestuft wird.

Wie in Abbildung 6.1 zu erkennen ist, unterscheiden sich beide Funktionen vor allem im Bereich kleiner Detektionszahlen. Während die Utility-Theory-Funktion Threat-Reports mit wenigen Detektionen bereits stark gewichtet, verzögert die Sigmoid-Funktion diesen Anstieg bis zum Wendepunkt n_0 . Für große n konvergieren beide Funktionen gegen $W_{\text{max}} = 3$, wodurch die Ausreißer-Dämpfung in beiden Fällen strukturell garantiert ist (siehe Kapitel 7.2.4 (Szenario 4 – Ausreißer-Report)).

Die Wahl zwischen beiden Funktionen wird im Rahmen von zukünftigen Arbeiten durch empirische Evaluierung der Aggregationsergebnisse auf realen Datensätzen getroffen (siehe Kapitel 8 (Diskussion und Forschungslücken)).

6.3 Implementierung des Attention-basierten Selektionsmodells

Die Gewichtungsfunktion wird für jeden Threat-Report separat berechnet, wodurch keine wechselseitigen Abhängigkeiten zwischen den einzelnen Threat-Reports entstehen. Dadurch eignen sich sowohl die Gewichtungsberechnung als auch die anschließende Aggregation grundsätzlich für eine parallele Verarbeitung über mehrere Threat-Reports hinweg.

Im Gegensatz zu frühen spektralen Ansätzen in der Graph-Verarbeitung sind keine rechenintensiven Matrixoperationen wie Inversionen oder Eigenwertzerlegungen erforderlich [16].

Die gewählten Parameter, wie beispielsweise der Steigungsfaktor k oder der Wendepunkt n_0 , dienen in der aktuellen Implementierung als Referenzwerte. Eine finale Optimierung dieser Hyperparameter erfolgt in zukünftigen Arbeiten durch eine empirische Evaluierung der Aggregationsergebnisse auf Basis realer Datensätze (siehe Kapitel 8 (Diskussion und Forschungslücken)).

- **Erzeugung fester Merkmalsvektoren:** Die Anforderung der Invarianz gegenüber variablen Eingabekardinalitäten wird durch die Funktion `aggregate_weighted_reports` (Listing 10.3 (`aggregation.py`)) realisiert. In Anlehnung an den Attention-Mechanismus ermöglicht dieser Ansatz die Verarbeitung einer beliebigen Anzahl von Eingangsgrößen, während gleichzeitig ein Merkmalsvektor mit invarianter Dimension (Fixed-Size Input) extrahiert wird [16]. Dies stellt die notwendige Kompatibilität für nachgelagerte ML-Modelle sicher.
- **Relevanzbasierte Gewichtung:** Die qualitative Differenzierung der Threat-Reports erfolgt in der Funktion `count_attack_ongoing_per_report` (Listing 10.6 (`attack_count.py`)). Diese extrahiert die Anzahl der Einzeldetektionen innerhalb eines Threat-Reports und überführt diese als quantitative Basis in den Gewichtungsprozess. Damit wird sichergestellt, dass die Aggregation die Informationsdichte der einzelnen Quellen berücksichtigt.
- **Asymptotisches Sättigungsverhalten:** Um eine Verzerrung der Ergebnisse durch statistische Ausreißer oder extrem hohe Detektionszahlen zu verhindern, wurde ein asymptotisches Sättigungsverhalten implementiert. Die Funktionen `add_report_weight` und `add_report_weight_sigmoid` (Listing 10.3 (`aggregation.py`)) sind so konzipiert, dass sie gegen einen definierten Grenzwert konvergieren (siehe Kapitel 6.2 (Auswahl der Gewichtungsfunktion)). Während die Utility-Theory-Funktion einen streng konkaven Verlauf aufweist, nutzt die Sigmoid-Funktion eine nichtlineare S-Kurve zur Stabilisierung der Gewichte (siehe Abbildung 6.1), ähnlich der in Graph Attention Networks (GATs) verwendeten Sättigungs- oder Aktivierungsmechanismen [12, 16]. Eine vergleichende Evaluation beider mathematischen Modelle hinsichtlich ihrer Leistungsfähigkeit im operativen Betrieb ist Bestandteil zukünftiger Forschung (siehe Kapitel 8 (Diskussion und Forschungslücken)).

7 Funktionsverifikation

Zur Überprüfung der implementierten Aggregations- und Filtermechanismen wird das System in diesem Kapitel anhand eines strukturierten Versuchsaufbaus, mehrerer Testszenarien sowie einer Analyse des Echo-Chamber-Effekts untersucht.

7.1 Versuchsaufbau

Um die implementierten Aggregations- und Filtermechanismen des ReSiLENT-Systems praktisch zu überprüfen, wird das System in diesem Kapitel verschiedenen synthetischen Angriffs- und Fehlerszenarien ausgesetzt. Die Ergebnisse der Simulationen dienen der Validierung und dem direkten Vergleich der mathematischen Modelle.

Um das System evaluieren zu können, wurden beispielhafte Threat-Reports generiert, die ein breites Spektrum potenzieller Szenarien abdecken. Wie in Kapitel 3.3 (Detektionseinheiten der Ladeinfrastruktur) dargelegt, werden Flooding-Attacken durch einen Großteil der implementierten Detektionsmechanismen erkannt. Für die Testgenerierung wurde daher eine UDP-Flooding-Attacke mittels des Tools `hping3` durchgeführt:

```
hping3 -flood -udp -p 80 <IP-Adresse der Wallbox>
```

Dabei reagierte lediglich die *Power Temperature Detection* nicht, da diese ausschließlich physikalische Parameter wie Temperatur und Leistung überwacht, keine netzwerkspezifischen Daten analysiert und diese Parameter bei einer Flooding-Attacke keine aussagekräftige Veränderung aufweisen. Für die folgenden Szenarien wurden sechs Threat-Reports generiert (siehe Abbildung 7.4), die variierende Feature-Werte und leicht veränderte Einzeldetektionsergebnisse aufweisen.

	key	original_name	n_attacks	weight
0	report_00002	threat_report_attack.csv	0	1.000000
1	report_00003	threat_report_attack4_aka_benign.csv	6	1.902377
2	report_00004	threat_report_attack5_aka_benign.csv	6	1.902377
3	report_00005	threat_report_attack_3.csv	7	2.006829
4	report_00006	threat_report_attack_2.csv	8	2.101342

Abbildung 7.1: Output der Aggregation von fünf Threat-Reports (Utility-Theory-Funktion)

Zur besseren Evaluation der Code-Struktur werden die relevantesten Parameter bei jeder Ausführung protokolliert und ausgegeben.

In den folgenden Abbildungen werden die kumulierten Werte des Threat-Reports verdeutlicht, darunter statistische Kennzahlen wie Durchschnitt (Avg), Minimum (Min), Maximum (Max), Median, Anstieg (Increase) sowie die Gesamtsumme. Die Analyse erfolgt über Sliding Windows, die vom aktuellen Zeitpunkt (Offset 0s) bis zu einem Rückblick von $-120s$ reichen (siehe Kapitel 4.1 (Konzeption des Threat-Reports)). Im vorliegenden Fall ist ersichtlich, dass die *Attack Probability* der Hardware-Detektion mit einer Wahrscheinlichkeit von über 0,98 identifiziert wurde. Diese sinkt im aktuellsten Zeitfenster (Offset 0s) signifikant auf einen Wert unter 0,01 ab. Dies deutet darauf hin, dass das Angriffsereignis entweder beendet oder erfolgreich abgewehrt wurde.

metric_name	instance	offset	start_time	end_time	num_samples	latest_value	avg	min	max	median	increase	sum
ids_power_bus_voltage	ids_power:2113	0s	2026-03-23T16:07:20.9	2026-03-23T16:08:20.9	3	4.853	4.878333	4.853	4.901	4.881	0.02	14.635
ids_power_bus_voltage	ids_power:2113	-30s	2026-03-23T16:06:50.9	2026-03-23T16:07:50.9	3	4.901	4.907667	4.881	4.941	4.901	0.02	14.723
ids_power_bus_voltage	ids_power:2113	-60s	2026-03-23T16:06:20.9	2026-03-23T16:07:20.9	3	4.881	4.914333	4.881	4.941	4.921	0.02	14.743
ids_hardware_attack_probability	hardware_events:2116	0s	2026-03-23T16:07:20.9	2026-03-23T16:08:20.9	3	0.005355	0.661544	0.00536	0.98970	0.98953	0.00017	1.984632
ids_hardware_attack_probability	hardware_events:2116	-30s	2026-03-23T16:06:50.9	2026-03-23T16:07:50.9	3	0.989723	0.988976	0.98765	0.98970	0.989530	0.002072	2.966928
ids_hardware_attack_probability	hardware_events:2116	-60s	2026-03-23T16:06:20.9	2026-03-23T16:07:20.9	3	0.989553	0.822489	0.49026	0.98960	0.9876510	0.499289	2.467468

Abbildung 7.2: Übersicht der generierten Threat-Report-Szenarien

7.2 Szenariobasierte Funktionsvalidierung

Die nachfolgenden Szenarien dienen der praktischen Überprüfung des Systemverhaltens, wobei sowohl die Robustheit gegenüber fehlerhaften Eingangsdaten als auch das mathematische Grenzverhalten der Gewichtungsfunktionen verifiziert werden.

7.2.1 Szenario 1 – NaN-Werte (Fehlerbehandlung)

In diesem Szenario muss sichergestellt werden, dass die Generierung von Threat-Reports auch bei unvollständigen oder fehlerhaften Daten robust bleibt. So kann es im laufenden Betrieb vorkommen, dass aufgrund von Netzwerkproblemen oder Sensorausfällen von Exportern keine Daten mehr über Prometheus bezogen werden können (siehe Kapitel 3 (Details zur Systemarchitektur)).

Die Entscheidung zum vollständigen Ausschluss solcher Threat-Reports basiert auf der Tatsache, dass ein Threat-Report ohne valide Daten keine Informationsgrundlage bietet, um den Angriffsstatus der Wallbox zu beurteilen. Eine Substitution der NaN-Werte durch 0 oder andere Standardwerte wurde als nicht sinnvoll erachtet, da dies das Ergebnis der Bedrohungsanalyse verfälschen könnte [5].

Es wurde eine Funktion implementiert, welche die generierten Threat-Reports auf NaN-Werte überprüft und diese gegebenenfalls ausschließt (siehe Listing 10.4, Funktion `filter_reports_with_NaN`). Aufgrund dieser Implementierung wird erwartet, dass ein Threat-Report mit resultierenden NaN-Werten bereits vor der Aggregation ausgeschlossen wird und der Nutzer über diesen Ausschluss informiert wird.

Um dieses Szenario zu erzeugen, wurde während der kontinuierlichen Threat-Reporterstellung ein Datenexporter terminiert und neugestartet. Dies führte zu einer erhöhten Anzahl an NaN-Werten in den generierten Threat-Reports, da Feature-Werte über Prometheus nicht mehr bezogen werden konnten (siehe Kapitel 3 (Details zur Systemarchitektur)).

metric_name	instance	offset	start_time	end_time	num	latest_value	avg	min	max	median	increase	sum
ids_hardware_feature_bus_access	hardware_events:2116	0s	2026-03-23T16:07:04.9	2026-03-23T16:08:04.9	3	95311	86768	70130	95311	94863	25181	260304
ids_hardware_feature_bus_access	hardware_events:2116	-30s	2026-03-23T16:06:34.9	2026-03-23T16:07:34.9	3	94863	NaN	NaN	NaN	70130	24733	NaN
ids_hardware_feature_bus_access	hardware_events:2116	-60s	2026-03-23T16:06:04.9	2026-03-23T16:07:04.9	3	70130	NaN	NaN	NaN	NaN	0	NaN

Abbildung 7.3: Auszug eines Threat-Reports aus Szenario 1 mit NaN-Werten

Infolgedessen wurde der betroffene Threat-Report bereits vor der Aggregation ausgeschlossen.

```

ataAggregator$ python -m cti_agg.pipeline
/home/yannic/Desktop/Master_rbg/Semester1/Forschungsarbeit1/code/EVSE_Testumgebung/DataAggregator
/cti_agg/pipeline.py:18: UserWarning: report_00001 (threat_report_nan_value.csv) has NaN value(s)
and got ignored
manifest = filter_reports_with_NaN(manifest, h5_path=h5_path)
  key          original_name  n_attacks  weight
0  report_00002      threat_report_attack.csv          9  2.186861
1  report_00003  threat_report_attack4_aka_benign.csv      6  1.902377
2  report_00004  threat_report_attack5_aka_benign.csv      6  1.902377
3  report_00005      threat_report_attack_3.csv          7  2.006829
4  report_00006      threat_report_attack_2.csv          8  2.101342

```

Abbildung 7.4: Auszug der Aggregation aus Szenario 1 mit NaN-Werten (Utility-Theory-Funktion)

Dieser Auszug der Aggregation bestätigt das erwartete Ergebnis.

7.2.2 Szenario 2 – Verschobene Zeitreihe

Eine Aggregation von Threat-Reports unterschiedlicher Knoten kann zu zeitlichen Verschiebungen führen, da die Knoten ihre Ergebnisse variabel über den Pub/Sub-Kanal an ihre Nachbarn weitergeben. Dies kann zu einer verzerrten Darstellung der Bedrohungsanalyse führen, da die Merkmale unterschiedliche Systemzustände repräsentieren und keine konsistente Informationsbasis bilden. Die Auswirkungen zeitlicher Verschiebungen auf die Aggregationsqualität sowie die optimale Wahl des Schwellenwerts `MAX_TIME_SKEW_SECONDS` sind Gegenstand empirischer Evaluierungen (siehe Kapitel 8 (Diskussion und Forschungslücken)).

Das erwartete Ergebnis ist, dass der betroffene Threat-Report bereits vor der Aggregation ausgeschlossen und der Nutzer über diesen Ausschluss informiert wird. Um dies zu evaluieren, wurde einem der sechs Threat-Reports eine zeitliche Verschiebung von +3 Minuten hinzugefügt. Dies überschreitet den definierten Grenzwert von `MAX_TIME_SKEW_SECONDS = 90s`, weshalb dieser Threat-Report ignoriert und der Nutzer darüber informiert werden soll.

```

ataAggregator$ python -m cti_agg.pipeline
/home/yannic/Desktop/Master_rbg/Semester1/Forschungsarbeit1/code/EVSE_Testumgebung/DataAggregator
/cti_agg/pipeline.py:18: UserWarning: report_00001 (threat_report_nan_value.csv) has NaN value(s)
and got ignored
manifest = filter_reports_with_NaN(manifest, h5_path=h5_path)
/home/yannic/Desktop/Master_rbg/Semester1/Forschungsarbeit1/code/EVSE_Testumgebung/DataAggregator
/cti_agg/pipeline.py:19: UserWarning: report_00006 (threat_report_attack_2.csv) time skew 139.3s
> 90s; got ignored
manifest = filter_reports_by_time_skew(manifest, h5_path=h5_path)
  key          original_name  n_attacks  weight
0  report_00002      threat_report_attack.csv          9  1.978026
1  report_00003  threat_report_attack4_aka_benign.csv      6  1.342877
2  report_00004  threat_report_attack5_aka_benign.csv      6  1.342877
3  report_00005      threat_report_attack_3.csv          7  1.515909

```

Abbildung 7.5: Auszug der Aggregation aus Szenario 2 mit zeitlicher Verschiebung (Sigmoid-Funktion)

Dieser Auszug der Aggregation bestätigt, dass Threat-Reports mit verschobenem Zeitfenster bereits vor der Aggregation ausgeschlossen werden und der Nutzer über diesen Ausschluss informiert wird.

7.2.3 Szenario 3 – Uniforme negative Detektionen

Wie in Kapitel 6.2 (Auswahl der Gewichtungsfunktion) erläutert, wird jedem Threat-Report eine Basisgewichtung von $W_{\text{base}} = 1$ zugewiesen, sofern keine positiven Detektionen vorliegen. In

diesem Fall wird erwartet, dass die Ergebnisse der Sigmoid-Funktion und der Utility-Theory-Funktion identisch sind, da beide Funktionen bei einer Anzahl von $n = 1$ positiven Detektionen (Referenzwert) auf den Wert 1 normiert sind.

Um dieses Szenario zu validieren, wurden Threat-Reports generiert, während sich die Ladeinfrastruktur in einem regulären Betriebszustand (*benign*) befand. Dies resultiert in durchgehend negativen Detektionswerten (0).

$$w(n) = W_{\text{base}} + (W_{\text{max}} - W_{\text{base}}) \cdot (1 - e^{-K \cdot (n-1)}) \quad (7.1)$$

$$w(n) = W_{\text{base}} + (W_{\text{max}} - W_{\text{base}}) \cdot \left(\frac{1}{1 + e^{-k(n-n_0)}} - \frac{1}{1 + e^{-k(1-n_0)}} \right) \quad (7.2)$$

Da für $n = 1$ beide Formeln den Wert 1 liefern, entspricht dies dem erwarteten Output in der Zusammenfassung des Aggregierten-Threat-Reports.

```
ataAggregator$ python -m cti_agg.pipeline
```

	key	original_name	n_attacks	weight
0	report_00001	threat_report3.csv	0	1.0
1	report_00002	threat_report4.csv	0	1.0
2	report_00003	threat_report0.csv	0	1.0
3	report_00004	threat_report2.csv	0	1.0
4	report_00005	threat_report1.csv	0	1.0

Abbildung 7.6: Auszug eines Aggregierten-Threat-Reports (Utility Theory) aus Szenario 3

```
ataAggregator$ python -m cti_agg.pipeline
```

	key	original_name	n_attacks	weight
0	report_00001	threat_report3.csv	0	1.0
1	report_00002	threat_report4.csv	0	1.0
2	report_00003	threat_report0.csv	0	1.0
3	report_00004	threat_report2.csv	0	1.0
4	report_00005	threat_report1.csv	0	1.0

Abbildung 7.7: Auszug eines Aggregierten-Threat-Reports (Sigmoid) aus Szenario 3

Die Zusammenfassungen der Aggregierten-Threat-Reports verdeutlichen, dass für jeden Threat-Report die identische Gewichtung von 1 berechnet wurde.

metric_name	instance	offset	start_time	end_time	num_sample	latest_value	avg	min	max	median	increase	sum	n_reports_included
ids_power_bus_voltage	ids_power:21130s		2026-03-23T16:07:04	2026-03-23T16:08:04	3	0	4.915667	4.889	4.961	4.897	0.072	14.747	5
ids_power_bus_voltage	ids_power:21130s		2026-03-23T16:06:34	2026-03-23T16:07:34	3	0	4.918333	4.889	4.961	4.905	0.072	14.755	5
ids_power_bus_voltage	ids_power:21130s		2026-03-23T16:06:04	2026-03-23T16:07:04	3	0	4.918333	4.889	4.961	4.905	0	14.755	5

Abbildung 7.8: Übersicht Aggregierten-Threat-Reports Utility-Theory-Funktion (Szenario 3)

metric_name	instance	offset	start_time	end_time	num_sample	latest_value	avg	min	max	median	increase	sum	n_reports_included
ids_power_bus_voltage	ids_power:21130s		2026-03-23T16:07:04	2026-03-23T16:08:04	3	0	4.915667	4.889	4.961	4.897	0.072	14.747	5
ids_power_bus_voltage	ids_power:21130s		2026-03-23T16:06:34	2026-03-23T16:07:34	3	0	4.918333	4.889	4.961	4.905	0.072	14.755	5
ids_power_bus_voltage	ids_power:21130s		2026-03-23T16:06:04	2026-03-23T16:07:04	3	0	4.918333	4.889	4.961	4.905	0	14.755	5

Abbildung 7.9: Übersicht Aggregierten-Threat-Reports Sigmoid-Funktion (Szenario 3)

Daraus lässt sich schließen, dass Threat-Reports mit einer gleichen Anzahl an positiven Einzeldektionen innerhalb des Systems als gleichwertig eingestuft und mit der selben Relevanz gewichtet werden und somit auch die selben Aggregierten Ergebnisse liefern (siehe Abbildung 7.8, 7.9).

7.2.4 Szenario 4 – Ausreißer-Report

Um nachzuweisen, dass die Gewichtung auch bei extremen Ausreißern den Grenzwert $W_{\max} = 3$ nicht überschreitet, wird das Grenzverhalten (Limes) für $n \rightarrow \infty$ betrachtet.

1. Exponentielle Utility-Funktion:

Die Sättigung der exponentiellen Funktion gegen den Grenzwert lässt sich wie folgt herleiten:

$$\begin{aligned}
 \lim_{n \rightarrow \infty} w(n) &= \lim_{n \rightarrow \infty} [1 + (3 - 1) \cdot (1 - e^{-0,1 \cdot n - 1})] \\
 &= 1 + 2 \cdot \left(1 - \underbrace{\lim_{n \rightarrow \infty} e^{-0,1 \cdot n - 1}}_{\rightarrow 0, \text{ da } -0,1n - 1 \rightarrow -\infty} \right) \\
 &= 1 + 2 \cdot (1 - 0) \\
 &= 3
 \end{aligned} \tag{7.3}$$

2. Angepasste Sigmoid-Funktion:

Analog dazu wird die Sättigung für die Sigmoid-Funktion berechnet:

$$\begin{aligned}
 \lim_{n \rightarrow \infty} w(n) &= \lim_{n \rightarrow \infty} \left[W_{\text{base}} + (W_{\max} - W_{\text{base}}) \underbrace{\left(\frac{1}{1 + e^{-k(n-n_0)}} - \frac{1}{1 + e^{-k(1-n_0)}} \right)}_{\text{Sigmoid-Differenz}} \right] \\
 &= W_{\text{base}} + (W_{\max} - W_{\text{base}}) \left(\underbrace{\lim_{n \rightarrow \infty} \frac{1}{1 + e^{-k(n-n_0)}}}_{\rightarrow 1, \text{ da } e^{-k(n-n_0)} \rightarrow 0} - \frac{1}{1 + e^{-k(1-n_0)}} \right) \\
 &= W_{\text{base}} + (W_{\max} - W_{\text{base}}) \cdot \frac{e^{-k(1-n_0)}}{1 + e^{-k(1-n_0)}} \\
 &= 1 + 2 \cdot \frac{e^{4,5}}{1 + e^{4,5}} \\
 &\approx 2,978
 \end{aligned} \tag{7.4}$$

7.3 Analyse des Echo-Chamber-Effekts

Ergänzend zur szenariobasierten Funktionsvalidierung wird in diesem Abschnitt der Echo-Chamber-Effekt als potenzielle systemische Grenze des Aggregationsansatzes analysiert.

7.3.1 Definition und theoretischer Kontext

Der Echo-Chamber-Effekt beschreibt im Kontext kollaborativer Intrusion Detection Systeme ein Phänomen, bei dem eine Mehrheit fehlerhafter oder kompromittierter Knoten durch korrelierende Fehlmeldungen den aggregierten Bedrohungsstatus des Gesamtsystems signifikant verzerrt. Debar und Wespi [3] demonstrieren, dass eine unkontrollierte Aggregation von Alarmen (Alerts) zu einer potenziell verfälschten Sicherheitsbewertung führen kann. Übertragen auf das vorliegende P2P-Szenario impliziert dies: Sofern eine signifikante Menge der Knoten fälschlicherweise einen Angriff meldet, signalisiert der aggregierte Threat-Report einen Angriffsstatus, obwohl korrekt messende Knoten zeitgleich einen benign Zustand detektieren.

7.3.2 Formalisierung des Szenarios

Sei M die Anzahl fehlerhafter Knoten, die fälschlicherweise einen Angriff (Status 1) melden, und K die Anzahl korrekter Knoten, welche einen benign Zustand (Status 0) identifizieren. Der aggregierte Output agg berechnet sich aus der gewichteten Summe der individuellen Threat-Reports gemäß der Aggregationsfunktion `aggregate_weighted_reports` (siehe Listing 10.3 (`aggregation.py`)):

$$agg = \frac{\sum_i report_i \cdot weight_i}{\sum_i weight_i} \quad (7.5)$$

mit:

- agg : aggregierter Wert eines einzelnen Features im *Aggregierten_Threat_Report*
- $report_i$: Feature-Wert des i -ten Threat-Reports
- $weight_i$: Gewichtung des i -ten Threat-Reports

Sobald das gewichtete Verhältnis von M zu K einen kritischen Schwellenwert überschreitet, divergiert der Aggregatwert in Richtung des Angriffsstatus, ungeachtet der konsistenten Benign-Meldungen der K korrekten Knoten.

- **Einfluss der Utility-Theory-Funktion:** Unter Anwendung der Utility-Theory-Funktion erzielen fehlerhafte Knoten bereits bei einer geringen Anzahl positiver Detektionen eine vergleichsweise hohe Gewichtung (siehe Abbildung 6.1). Dies ist auf den konkaven Verlauf der Funktion zurückzuführen, die ihren steilsten Anstieg bei $n = 1$ aufweist. Infolgedessen kann bereits eine moderate Anzahl fehlerhafter Knoten mit mittlerer Detektionsrate den Aggregatwert entscheidend verzerrten. Das System erweist sich unter dieser Funktion somit als besonders anfällig gegenüber dem Echo-Chamber-Effekt bei moderaten M/K -Verhältnissen.

- **Einfluss der Sigmoid-Funktion:** Die Sigmoid-Funktion dämpft diesen Effekt durch die Einführung eines Wendepunkts n_0 . Fehlerhafte Knoten mit wenigen positiven Einzeldetektionen erhalten zunächst eine geringe Gewichtung und erreichen erst nach Überschreiten des Schwellenwerts n_0 signifikanten Einfluss (siehe Abbildung 6.1). Dennoch wirken sich extreme Ausreißer Knoten mit einer sehr hohen Anzahl an Fehldetektionen ähnlich gravierend aus wie bei der Utility-Theory-Funktion, da beide Funktionen asymptotisch gegen das Maximum $W_{max} = 3$ konvergieren (siehe Kapitel 7.2.4 Szenario 4 – Ausreißer-Report).

7.3.3 Einordnung und Ausblick

Es ist festzustellen, dass keine der untersuchten Funktionen das Echo-Chamber-Problem vollständig zu lösen vermag. Beide Ansätze begrenzen zwar durch W_{max} den maximalen Einfluss eines einzelnen Threat-Reports, eliminieren jedoch nicht die strukturelle Verzerrung durch eine korrelierte Mehrheit. Diese Problematik wird als offene Forschungsfrage in Kapitel 8 (Diskussion und Forschungslücke) weiterführend thematisiert.

8 Diskussion und Forschungslücken

Nach der Vorstellung der theoretischen und praktischen Ergebnisse werden die gewählten Ansätze im Folgenden kritisch bewertet. Dieses Kapitel analysiert die konkreten Auswirkungen der Gewichtungsstrategien und identifiziert zentrale Variablen sowie offene Forschungsfragen für zukünftige Arbeiten.

8.1 Auswirkungen der Gewichtungsstrategien auf die Threat-Report-Qualität

Die im Kapitel 6.2 (Auswahl der Gewichtungsfunktion) angepassten Funktionen erfüllen die in Kapitel 6.1 (Anforderungen an die Gewichtungsfunktion) definierten Anforderungen. Sie unterscheiden sich jedoch strukturell in ihrem Verhalten bei niedrigen positiven Einzeldetektionsergebnissen, was unmittelbare Auswirkungen auf die Robustheit der Aggregation haben kann.

Die Utility-Theory-Funktion (siehe Gleichung 6.1) basiert auf dem Prinzip des abnehmenden Grenznutzens (*Diminishing Returns*), formalisiert durch die *Negative Exponential Utility Function* mit konstanter absoluter Risikoaversion (CARA) [10, Sec. 8, Sec. 9]:

Die Utility-Funktion ist rein konkav: Der stärkste Gewichtungsanstieg findet bereits bei $n = 1$ statt (siehe Abbildung 6.1), was dazu führt, dass Threat-Reports mit wenigen Einzeldetektionen bereits signifikant gewichtet werden [10]. Dies birgt das Risiko einer erhöhten Anfälligkeit gegenüber Rauschen oder dem Echo-Chamber-Effekt (siehe Kapitel 7.3 (Analyse des Echo-Chamber-Effekts)), da auch vereinzelte, potenziell fehlerhafte Detektionen einen starken Einfluss auf die Aggregation ausüben (siehe Kapitel 7.2 (Szenariobasierte Funktionsvalidierung)).

Die angepasste Sigmoid-Funktion (siehe Gleichung 6.2) ist ursprünglich als Aktivierungsfunktion in neuronalen Netzen und insbesondere in Graph Attention Networks zur Gewichtung von Nachbarschaftsinformationen etabliert [16]. Sie weist durch ihren S-förmigen Verlauf einen Wendepunkt bei n_0 auf (siehe Abbildung 6.1):

Unterhalb von n_0 steigt das Gewicht moderat an, erst nach Überschreiten dieser Schwelle wird ein Threat-Report als signifikant eingestuft. Dieses Verhalten modelliert implizit einen Mindestkonsensmechanismus und ist robuster gegenüber vereinzelt Fehl-detektionen als die Utility-Theory-Funktion.

Beide Funktionen konvergieren asymptotisch gegen $W_{\max} = 3$, wodurch die Dominanz statistischer Ausreißer strukturell begrenzt wird (siehe Kapitel 7.2.4 (Szenario 4 – Ausreißer-Report)). Der wesentliche Unterschied liegt im Bereich kleiner n_0 : Während die Utility-Theory-Funktion bereits einzelne Detektionen stark gewichtet, verzögert die Sigmoid-Funktion diesen Anstieg bis zum Erreichen des Wendepunkts n_0 .

Die Hyperparameter k , n_0 und $W_{\max}$ wurden in der vorliegenden Arbeit nicht empirisch optimiert, sondern als Referenzwerte festgesetzt. Eine systematische Evaluation auf realen Datensätzen ist daher Gegenstand zukünftiger Arbeiten.

8.2 Potenzial und Grenzen der Aggregation für die Detektionsqualität

Die kollaborative Aggregation von Threat-Reports zielt darauf ab, die Detektionsqualität einzelner Knoten durch die Einbeziehung kontextueller Informationen zu verbessern. Eine abschließende empirische Bewertung dieses Effekts ist jedoch nicht Gegenstand der vorliegenden Arbeit, sondern erfordert eine systematische Untersuchung auf Basis geeigneter Datensätze.

Stattdessen werden im Folgenden zentrale Einflussgrößen identifiziert, welche bereits in Listing 10.5 (`config.py`) ausgearbeitet wurden und welche Wirksamkeit diese in der Aggregation maßgeblich haben. Diese Faktoren bilden die Grundlage für zukünftige empirische Analysen.

8.2.1 Datenqualität vs. Datenmenge

Die implementierten Filtermechanismen `filterReportsWithNaN` und `filterReportsByTimeSkew` reduzieren die Anzahl eingehender Threat-Reports aktiv vor der Aggregation. Die zugrundeliegende These lautet: Ein vollständiger, zeitlich konsistenter Threat-Report trägt mehr zur Aggregationsqualität bei als eine größere Menge verrauschter oder unvollständiger Berichte [5]. Es bleibt offen, ab welchem Filteranteil der verbleibende Informationsgehalt nicht mehr ausreicht, um eine zuverlässige Aggregation zu gewährleisten.

8.2.2 Abtastintervall

Wie in Kapitel 4.1 (Zentralisiertes Datenmanagement und Merkmalsextraktion für den Threat-Report) beschrieben, erfasst der Threat-Reporter aktuell drei Datenpunkte bei $t = 0$ s, $t = -30$ s und $t = -60$ s. Die Wahl des 30-Sekunden-Intervalls und der 60-Sekunden-Zeitfenster stellt einen nicht empirisch validierten Kompromiss dar: Kürzere Intervalle erhöhen das Rauschen und die Auslastung im IPFS-Netzwerk, während längere Intervalle die Reaktionsfähigkeit auf laufende Angriffe verringern und veraltete Kontextinformation in die Aggregation einbringen [6, 19]. Der Einfluss variierender Intervalle auf die True-Positive-Rate des Gesamtsystems ist empirisch zu quantifizieren.

8.2.3 Verschobene Zeitfenster

Der Parameter `MAX_TIME_SKEW_SECONDS` ist aktuell auf 90 s hardcodiert (siehe Listing 10.4 (`filter.py`), 10.5 (`config.py`)). In einer zukünftigen Arbeit muss untersucht werden, welcher Schwellenwert die Bedrohungseinschätzung durch den Aggregierten-Threat-Report nicht verzerrt und wie die Gesamtergebnisse dadurch optimiert werden können.

8.2.4 Echo-Chamber-Effekt

Wie in Kapitel 7.3 (Analyse des Echo-Chamber-Effekts) formalisiert, bezeichnet der Echo-Chamber-Effekt das Szenario, in dem eine korrelierte Mehrheit kompromittierter Knoten den Aggregierten-Threat-Report systematisch verzerrt. Der definierte Grenzwert $W_{\max} = 3$ dämpft den Einfluss einzelner Ausreißer, eliminiert jedoch nicht die strukturelle Verzerrung durch eine gleichartig manipulierte Mehrheit. Zhang et al. [20] zeigen, dass insbesondere das Ungleichgewicht zwischen normalen und kompromittierten Knoten die Konstruktion eines akkuraten

Klassifikators erheblich erschwert. Zur Adressierung dieser Forschungslücke kommen folgende Ansätze in Betracht: eine reputationsbasierte Knotengewichtung über die Zeit (*Trust Score*), bei der der Reputationswert eines Knotens dessen Wahrscheinlichkeit repräsentiert, zuverlässige Informationen bereitzustellen, und kontinuierlich anhand seines Verhaltens im Netzwerk aktualisiert wird, sowie die Kreuzvalidierung Aggregierten-Threat-Reports mit den lokalen Detektionsergebnissen des empfangenden Knotens als unabhängigem Anker [20].

8.2.5 Evaluation auf öffentlichem Datensatz

Die Funktionsverifikation der vorliegenden Arbeit basiert ausschließlich auf einer physischen Testumgebung mit einer einzelnen ChargeIQ-Wallbox (siehe Kapitel 3 (Systemarchitektur und Detektion Framework)), was die externe Validität der Ergebnisse erheblich einschränkt. Ob das entwickelte Aggregations- und Gewichtungsverfahren auf Infrastrukturen außerhalb der eigenen Ladeumgebung generalisiert, bleibt ungeklärt. Als geeignete Evaluationsgrundlagen für zukünftige Arbeiten kommen etablierte IDS-Benchmarkdatensätze und IoT-spezifische Datensätze in Betracht. Erst eine solche Evaluation ermöglicht eine wissenschaftliche belastbare Aussage über den tatsächlichen Mehrwert des kollaborativen Ansatzes gegenüber rein lokaler Anomalieerkennung.

9 Zusammenfassung und Ausblick

Nachfolgend werden die Kernaspekte der Methodik sowie die Ergebnisse der Funktionsverifikation zusammengefasst.

9.1 Methodik und Implementierung

Im Rahmen dieser Arbeit wurde eine kollaborative Aggregationsinfrastruktur für das ReSiLENT-Detektionssystem konzipiert und implementiert. Jede Ladestation generiert standardisierte *Threat-Reports* aus vier spezialisierten Detektionseinheiten. Die Verteilung dieser Threat-Reports erfolgt über ein privates IPFS-Peer-to-Peer-Netzwerk.

Zur Aggregation variabler Mengen eingehender Threat-Reports wurde ein Aggregationsansatz implementiert, dessen Grundprinzip auf Deep-Sets und Attention-Mechanismen basiert. Dadurch kann eine variable Anzahl eingehender Threat-Reports in einen Merkmalsvektor fester Größe (*fixed-size representation*) überführt werden.

Die Relevanz einzelner Knoten wird dabei über zwei angepasste Gewichtungsfunktionen modelliert: eine modifizierte *Negative Exponential Utility Function* (CARA) sowie eine angepasste *Sigmoid-Funktion*. Beide Funktionen dienen der dynamischen Gewichtung der eingehenden Threat-Reports und begrenzen durch ihr asymptotisches Sättigungsverhalten den Einfluss statistischer Ausreißer.

9.2 Ergebnisse der Funktionsverifikation

Die Verifikation anhand synthetisch generierter Szenarien zeigt, dass beide Gewichtungsfunktionen die definierten Anforderungen erfüllen. Mit NaN-Werten behaftete oder zeitlich signifikant verschobene Threat-Reports werden zuverlässig vorgefiltert. Die Gewichtung konvergiert strukturell gegen den Grenzwert $W_{\max} = 3$, wodurch der Einfluss von Ausreißern effektiv gedämpft wird.

Bei uniformen Detektionswerten liefern beide Funktionen identische Ergebnisse. Differenzen treten primär im Bereich geringer Detektionszahlen auf, in dem die Utility-Funktion eine stärkere Gewichtung vornimmt als die Sigmoid-Funktion. Durch das asymptotische Sättigungsverhalten wird der *Echo-Chamber-Effekt* strukturell begrenzt, wenngleich eine vollständige Elimination allein durch die Gewichtung nicht erreicht wird.

9.3 Zukünftige Forschungsaspekte

Die empirische Validierung auf Basis eines öffentlichen und unabhängigen IDS-Datensatzes stellt den nächsten notwendigen Schritt dar, um den wissenschaftlichen Mehrwert des kollaborativen Ansatzes belastbar zu quantifizieren.

Im Zentrum zukünftiger Forschungsfragen steht die Optimierung der Hyperparameter, namentlich des Skalierungsfaktors k , der Basisanzahl n_0 , des Abtastintervalls sowie des Schwellenwerts `MAX_TIME_SKEW_SECONDS`.

Es wird die Hypothese aufgestellt, dass die Genauigkeit der Aggregation maßgeblich davon abhängt, wie aktuell die Threat-Reports sind und wie schnell das System diese verarbeiten kann (Latenz und Durchsatz). Daher ist anzunehmen, dass eine starre Einstellung der Parameter schlechtere Ergebnisse liefert als eine adaptive Konfiguration, die sich automatisch an den Zustand des jeweiligen Systems anpasst.

Darüber hinaus soll untersucht werden, inwieweit der *Echo-Chamber-Effekt* durch die Integration reputationsbasierter Trust-Scores nicht nur begrenzt, sondern gezielt kompensiert werden kann. Hierbei gilt es die Annahme zu prüfen, dass die Gewichtung der Informationsquelle (Reputation) eine robustere Metrik darstellt als die rein numerische Sättigung der Gewichtungsfunktionen.

10 Quellcode

Listing 10.1: reporter.py – Logik der Report-Generierung

```
import time
import requests
import csv
import statistics
from datetime import datetime, timedelta
from queries import get_queries # liefert NUR Basis-PromQL-Expressions zurück

# === KONFIGURATION ===
PROMETHEUS_URL = "http://192.168.0.25" # Prometheus-Endpunkt
IPFS_API_URL = "http://localhost:3001" # IPFS API-Endpunkt
OUTPUT_CSV = "threat_report.csv"

# Zeitraum
TIMESPAN = 1 # Minuten
END_TIME = datetime.utcnow()
START_TIME = END_TIME - timedelta(minutes=TIMESPAN)
STEP = "30s"

# Zusätzliche Zeitversätze (in Sekunden)
OFFSETS = [0, -30, -60]

def query_prometheus_range(query, start, end, step):
    """Fragt Prometheus nach einem Zeitbereich (range query)."""
    url = f"{PROMETHEUS_URL}/api/v1/query_range"
    params = {
        "query": query,
        "start": start.isoformat() + "Z",
        "end": end.isoformat() + "Z",
        "step": step,
    }
    response = requests.get(url, params=params)
    response.raise_for_status()
    result = response.json()

    if result["status"] != "success":
        raise RuntimeError(f"Fehler bei Abfrage: {result}")

    return result["data"]["result"]

def compute_basic_stats(values):
    """Berechne avg, min, max, median aus einer Liste von Floats."""
    if not values:
        return None
    try:
        avg = sum(values) / len(values)
        mn = min(values)
```

```

        mx = max(values)
        med = statistics.median(values)
    except Exception:
        return None

    return {"avg": avg, "min": mn, "max": mx, "median": med}

def compute_increase(values):
    """Berechnet eine einfache 'increase' Approximation aus einer Liste
        von Floats."""
    if len(values) < 2:
        return None
    inc = 0.0
    prev = values[0]
    for v in values[1:]:
        delta = v - prev
        if delta > 0:
            inc += delta
        prev = v
    return inc

def main():
    QUERIES = get_queries(START_TIME, END_TIME)

    with open(OUTPUT_CSV, mode="w", newline="", encoding="utf-8") as file
        :
        writer = csv.writer(file)
        # Kopfzeile
        writer.writerow([
            "metric_name",
            "instance",
            "offset",
            "start_time",
            "end_time",
            "num_samples",
            "latest_value",
            "avg",
            "min",
            "max",
            "median",
            "increase",
            "sum"
        ])

    for metric_name, promql in QUERIES.items():
        print(f"Metrik: {metric_name} | Query: {promql}")
        for offset in OFFSETS:
            offset_end = END_TIME + timedelta(seconds=offset)
            offset_start = offset_end - timedelta(minutes=TIMESPAN)

            series_list = query_prometheus_range(promql, offset_start

```

```

        , offset_end, STEP)

for series in series_list:
    metric_labels = series.get("metric", {})
    instance = metric_labels.get("instance", "unknown")

    raw_values = []
    for ts, value_str in series.get("values", []):
        try:
            v = float(value_str)
        except ValueError:
            continue
        raw_values.append(v)

    if not raw_values:
        continue

    stats = compute_basic_stats(raw_values)
    inc = compute_increase(raw_values)
    total_sum = sum(raw_values)
    latest_value = raw_values[-1] if raw_values else ""

    if stats is None:
        continue

    writer.writerow([
        metric_name,
        instance,
        f"{offset}s",
        offset_start.isoformat() + "Z",
        offset_end.isoformat() + "Z",
        len(raw_values),
        f"{latest_value:.6f}",
        f"{stats['avg']:.6f}",
        f"{stats['min']:.6f}",
        f"{stats['max']:.6f}",
        f"{stats['median']:.6f}",
        f"{inc:.6f}" if inc is not None else "",
        f"{total_sum:.6f}",
    ])

print(f"CSV-Datei erstellt: {OUTPUT_CSV}")

if __name__ == "__main__":
    main()
    #time.sleep(TIMESPAN) # Der Report wird nur einmal pro Minute
    aktualisiert
    #print("Warte auf nächste Aktualisierung...")
    #
    # Hier testweise eine testdatei veröffentlichen
    cid = publish_csv("./testfile.csv", IPFS_API_URL)

```


Listing 10.2: queries.py – Logik der Datenbankabfragen

```
from typing import Dict
from datetime import datetime

def get_queries(start_time: datetime, end_time: datetime) -> Dict[str,
str]:
    """Gibt NUR Basis-PromQL-Metriken zurück (KEINE Aggregationen)."""
    return {
        # System Status
        "attack_ongoing": "attack_ongoing",

        # Power Metrics (Basis)
        "ids_power_bus_voltage": "ids_power_bus_voltage_V",
        "ids_power_current": "ids_power_current_mA",
        "ids_power_power": "ids_power_power_mW",
        "ids_power_shunt_voltage": "ids_power_shunt_voltage_mV",
        "ids_power_lstm_mse": "ids_power_lstm_mse",
        "ids_power_isolation_score": "ids_power_isolation_score",

        # Temperature Metrics (Basis)
        "ids_temperature": "ids_temperature",
        "evse_thermo_current_temp": "evse_thermo_current_temp_celsius",

        # Wattmeter Status
        "ids_wattmeter": "ids_wattmeter",

        # Anomaly Detection
        "ids_power_isolation_forest_anomaly": "
            ids_power_isolation_forest_anomaly",
        "ids_power_lstm_anomaly": "ids_power_lstm_anomaly",

        # Transformer ML Predictions (Basis)
        "transformer_prediction": "transformer_prediction",
        "ids_transformer_attack_probability": "
            ids_transformer_attack_probability",
        "ids_transformer_shunt_voltage": "ids_transformer_shunt_voltage",
        "ids_transformer_bus_voltage": "ids_transformer_bus_voltage",
        "ids_transformer_current": "ids_transformer_current",
        "ids_transformer_power": "ids_transformer_power",
        "ids_transformer_raw_prediction": "ids_transformer_raw_prediction
            ",

        # xLSTM Metrics
        "ids_xlstm_prediction": "ids_xlstm_prediction",
        "ids_xlstm_attack_probability": "ids_xlstm_attack_probability",
        "ids_xlstm_reconstruction_loss": "ids_xlstm_reconstruction_loss",

        # Ensemble
        "ids_ensemble_final_decision": "ids_ensemble_final_decision",

        # Hardware Features (Basis)
        "ids_hardware_feature_node_stores": "
```

```

        ids_hardware_feature_node_stores",
"ids_hardware_feature_bus_access": "
        ids_hardware_feature_bus_access",
"ids_hardware_feature_skb_kfree_skb": "
        ids_hardware_feature_skb_kfree_skb",
"ids_hardware_feature_cpu_cycles": "
        ids_hardware_feature_cpu_cycles",
"ids_hardware_feature_l2d_cache_wb_victim": "
        ids_hardware_feature_l2d_cache_wb_victim",
"ids_hardware_feature_syscalls_sys_enter_fcntl": "
        ids_hardware_feature_syscalls_sys_enter_fcntl",
"ids_hardware_feature_sched_sched_stat_runtime": "
        ids_hardware_feature_sched_sched_stat_runtime",
"ids_hardware_feature_st_spec": "ids_hardware_feature_st_spec",
"ids_hardware_feature_syscalls_sys_enter_kill": "
        ids_hardware_feature_syscalls_sys_enter_kill",
"ids_hardware_feature_fib_fib_table_lookup": "
        ids_hardware_feature_fib_fib_table_lookup",
"ids_hardware_feature_sock_inet_sock_set_state": "
        ids_hardware_feature_sock_inet_sock_set_state",
"ids_hardware_feature_l2d_cache_wb": "
        ids_hardware_feature_l2d_cache_wb",
"ids_hardware_feature_unaligned_st_spec": "
        ids_hardware_feature_unaligned_st_spec",
"ids_hardware_feature_mem_access_rd": "
        ids_hardware_feature_mem_access_rd",
"ids_hardware_feature_l2d_cache_refill_wr": "
        ids_hardware_feature_l2d_cache_refill_wr",
"ids_hardware_feature_mmc_mmc_request_start": "
        ids_hardware_feature_mmc_mmc_request_start",
"ids_hardware_feature_br_mis_pred": "
        ids_hardware_feature_br_mis_pred",

# Hardware Attack Detection
"ids_hardware_attack_probability": "
        ids_hardware_attack_probability",
"ids_hardware_benign_probability": "
        ids_hardware_benign_probability",
"ids_hardware_attack_prediction": "ids_hardware_attack_prediction
    ",

# IoT Platform Metrics (neu hinzugefügt aus neuen Metriken)
"iotplatform_meter_bus_voltage": "iotplatform_meter_bus_voltage_V
    ",
"iotplatform_meter_current": "iotplatform_meter_current_mA",
"iotplatform_meter_power": "iotplatform_meter_power_mW",
"iotplatform_meter_shunt_voltage": "
        iotplatform_meter_shunt_voltage_mV",
}

```

Listing 10.3: aggregation.py – Implementierung der Gewichtungsfunktion und Aggregationslogik

```
from __future__ import annotations
```

```

from pathlib import Path
import pandas as pd
import numpy as np

from .config import BASE_WEIGHT, MAX_WEIGHT, K, BINARY_INDICATORS,
    BINARY_COLS, BINARY_THRESHOLD

def add_report_weight(
    df_counts: pd.DataFrame,
    n_col: str = "n_attacks",
    out_col: str = "weight",
) -> pd.DataFrame:
    """
    Berechnet Gewichte basierend auf angepassten UtilityTheory-Funktion:
    utilityTheory(n)=W_{base}+(W_{max}-W_{base}) (1-exp^{-(K (n))})
    """
    out = df_counts.copy()
    n = pd.to_numeric(out[n_col], errors="coerce").fillna(0).to_numpy() +
        1
    out[out_col] = BASE_WEIGHT + (MAX_WEIGHT - BASE_WEIGHT) * (1.0 - np.
        exp(-K * (n - 1)))
    return out

def add_report_weight_sigmoid(
    df_counts: pd.DataFrame,
    n_col: str = "n_attacks",
    out_col: str = "weight",
    n0: float = 10.0,    # Wendepunkt
    k: float = 0.5,      # Steilheit
    base_weight: float = 1.0,
    max_weight: float = 3.0,
) -> pd.DataFrame:
    """
    sigmoid(n) = base_weight + (max_weight - base_weight) *
        ( (1/(1+exp(-k(n-n0)))) - (1/(1+exp(-k(1-n0)))) )
    """
    out = df_counts.copy()
    n = pd.to_numeric(out[n_col], errors="coerce").fillna(0).to_numpy() +
        1

    sig_n = 1.0 / (1.0 + np.exp(-k * (n - n0)))

    # Korrektur-Term, damit die Kurve bei n=1 genau bei 0 startet
    sig_1 = 1.0 / (1.0 + np.exp(-k * (1.0 - n0)))

    # Skalierung auf den Bereich zwischen BASE_WEIGHT und MAX_WEIGHT
    out[out_col] = base_weight + (max_weight - base_weight) * (sig_n -
        sig_1)

```

```

# verhindert Werte < base_weight
out[out_col] = out[out_col].clip(lower=base_weight)

return out

def aggregate_weighted_reports(
    weights_df: pd.DataFrame,
    h5_path: str | Path = "reports.h5",
    key_col: str = "key",
    weight_col: str = "weight",
    cols: tuple[str, ...] = ("latest_value", "avg", "min", "max", "median",
                             "increase", "sum"),
) -> pd.DataFrame:
    """
    Aggregiert report_00001, report_00002, ... zu einem neuen Report:
    für jede Zelle der 'cols':
        
$$\text{agg} = (\sum \text{report}_i * \text{weight}_i) / (\sum \text{weight}_i)$$


    Alle Reports müssen gleiche Zeilenanzahl haben und gleiche Index/
    Zeilenreihenfolge).
    """
    h5_path = Path(h5_path)

    num = None # Numerator:  $\sum(df * w)$ 
    denom = 0.0

    validate_same_metrics(weights_df, h5_path=h5_path, key_col=key_col)

    for _, r in weights_df.iterrows():
        key = r[key_col]
        w = float(r[weight_col])

        df = pd.read_hdf(h5_path, key=key) # lädt DataFrame unter dem
        Key.
        x = df.loc[:, list(cols)].apply(pd.to_numeric, errors="coerce")

        if num is None:
            num = x * w
        else:
            # addiert elementweise; Align passiert über Index/Columns (
            muss matchen).
            num = num.add(x * w, fill_value=0)

        denom += w

    if num is None:
        raise ValueError("weights_df ist leer, nichts zu aggregieren.")
    if denom == 0:
        raise ValueError("Summe der Weights ist 0, Division nicht möglich
        .")

    agg = num / denom
    return agg

```

```

def validate_same_metrics(
    weights_df: pd.DataFrame,
    h5_path: str | Path,
    key_col: str = "key",
) -> None:
    """
    Prüft das nur aggregiert wird wenn alle Reports die gleichen
    metric_name Zeilen haben (gleicher Index, gleiche Reihenfolge).
    """
    h5_path = Path(h5_path)

    ref_index = None
    ref_key = None

    for _, r in weights_df.iterrows():
        key = r[key_col]
        df = pd.read_hdf(h5_path, key=key)

        if ref_index is None:
            ref_index = df.index
            ref_key = key
            continue

        if not ref_index.equals(df.index):
            raise ValueError(
                f"Metric_{key} has different metric_name/Order_
                as_{ref_key}."
            )

def binarize_indicators(aggregated_values: pd.DataFrame) -> pd.DataFrame:
    """
    Für die in BINARY_INDICATORS gelisteten Zeilen (Index) werden die
    Werte in BINARY_COLS anhand von BINARY_THRESHOLD binarisiert:
    Wenn Wert >= BINARY_THRESHOLD folgt 1, sonst 0.
    Damit Attack Indicators keine Komma Werte haben.
    """
    out = aggregated_values.copy()

    mask = out.index.astype(str).isin(BINARY_INDICATORS)
    if not mask.any():
        return out

    cols = [c for c in BINARY_COLS if c in out.columns]
    out.loc[mask, cols] = (out.loc[mask, cols] >= BINARY_THRESHOLD).
        astype("int64")
    return out

```

Listing 10.4: filters.py – Logik der Filterung von Threat-Reports

```

from __future__ import annotations

from pathlib import Path

```

```

import pandas as pd
import warnings

from .config import TIME_COL, MAX_TIME_SKEW_SECONDS

def filter_reports_with_NaN(
    manifest: pd.DataFrame,
    h5_path: str | Path,
    key_col: str = "key",
) -> pd.DataFrame:
    '''
    Wenn NaN werte irgendwo in einem report auftauchen wird er ignoriert
    '''

    h5_path = Path(h5_path)

    keep = []
    for _, row in manifest.iterrows():
        key = row[key_col]
        original_name = row.get("original_name", None)

        df = pd.read_hdf(h5_path, key=key)

        # True wenn irgendwo im gesamten Report ein NaN steht (auch
        # metric_name etc.)
        has_nan = df.isna().any(axis=None)
        if has_nan:
            warnings.warn(
                f"{key}_{(original_name)}_has_NaN_value(s)_and_got_ignored",
                category=UserWarning,
                stacklevel=2,
            )
            continue

        keep.append(row)

    return pd.DataFrame(keep).reset_index(drop=True)

def filter_reports_by_time_skew(
    manifest: pd.DataFrame,
    h5_path: str | Path,
    key_col: str = "key",
) -> pd.DataFrame:
    '''
    Wenn der Median des Timewindows eines reports mehr als
    MAX_TIME_SKEW_SECONDS von der Referenzzeit (Median über alle
    Reports) abweicht, wird er ignoriert.
    '''

    h5_path = Path(h5_path)

    # 1) Pro Report eine repräsentative Zeit bestimmen (Median der

```

```

    start_time-Spalte)
per_report_time: list[tuple[int, str, pd.Timestamp, str | None]] = []
for i, row in enumerate(manifest.itertuples(index=False)):
    key = getattr(row, key_col)
    original_name = getattr(row, "original_name", None)

    # load report
    df = pd.read_hdf(h5_path, key=key)
    # konvertiere in datetime werte
    t = pd.to_datetime(df[TIME_COL], utc=True, errors="coerce")
    # Berechne median pro report
    per_report_time.append((i, key, t.median(), original_name))

if not per_report_time:
    return manifest.iloc[0:0].copy()

# 2) Referenzzeit: Median über alle Reports
ref = pd.Series([x[2] for x in per_report_time]).median()

# 3) Filter: Abweichung <= MAX_TIME_SKEW_SECONDS
keep_rows = []
for i, key, t_med, original_name in per_report_time:
    skew = abs((t_med - ref).total_seconds())
    if skew > MAX_TIME_SKEW_SECONDS:
        warnings.warn(
            f"{key}_{original_name}_time_skew_{skew:.1f}s > {MAX_TIME_SKEW_SECONDS}s; got ignored",
            UserWarning,
            stacklevel=2,
        )
        continue
    keep_rows.append(manifest.iloc[i])

return pd.DataFrame(keep_rows).reset_index(drop=True)

```

Listing 10.5: config.py – Einstellbare Parameter

```

from __future__ import annotations
from pathlib import Path

MODULE_DIR = Path(__file__).resolve().parent

# Einheitliche Ordner
TMP_DIR = MODULE_DIR / "reports_tmp"

# TIME_Window bei versetzten zeiten
TIME_COL = "start_time"
MAX_TIME_SKEW_SECONDS = 90
WINDOW_SECONDS = 60

# Detection vorhersagen:
ATTACK_INDICATORS: tuple[str, ...] = (

```

```

        "attack_ongoing",
        "ids_hardware_attack_prediction",
        "ids_xlstm_attack_probability",
        "ids_transformer_attack_probability",
        "transformer_prediction",
        "ids_power_lstm_anomaly",
        "ids_wattmeter",
        "ids_power_isolation_forest_anomaly"
    )

    ATTACK_VALUE_COL: str = "latest_value"

    # Gewichtungs-Parameter
    BASE_WEIGHT: float = 1.0
    MAX_WEIGHT: float = 3.0
    K: float = 0.1

    # HDF5-Store Verhalten
    H5_FILENAME: str = "reports.h5"

    # Aggregation / Export
    AGG_COLS: tuple[str, ...] = (
        "latest_value", "avg", "min", "max", "median", "increase", "sum"
    )
    AGG_OUT_CSV_NAME: str = "aggregated_threat_report.csv"

    # Attack Indicators dürfen keine komma werte haben
    BINARY_INDICATORS: tuple[str, ...] = ATTACK_INDICATORS
    BINARY_COLS: tuple[str, ...] = AGG_COLS
    BINARY_THRESHOLD: float = 0.5 # >=0.5 => 1 sonst 0

```

Listing 10.6: attack_count.py – Zählt positive Einzeldetektionen

```

from __future__ import annotations

from pathlib import Path
import pandas as pd

from .config import ATTACK_INDICATORS, ATTACK_VALUE_COL


def count_attack_ongoing_per_report(
    manifest: pd.DataFrame,
    h5_path: str | Path = "reports.h5",
) -> pd.DataFrame:
    """
    Für jeden Report (manifest['key']):
    - Report aus HDF5 lesen
    - Zeile 'needle' finden (Index==needle oder irgendwo in der Zeile als
      Zellwert)
    - Wenn value_col == 1 -> n_attacks += 1
    Ergebnis: pro Report genau ein Counter n_attacks.
    """

```



```

h5_path = Path(h5_path)

out = []
for _, row in manifest.iterrows():
    key = row["key"]
    original_name = row.get("original_name", None)

    df = pd.read_hdf(h5_path, key=key)

    n_attacks = 0

    for needle in ATTACK_INDICATORS:
        # Zeile finden: entweder Index == needle oder irgendwo in
        # einer Zeile als Zellwert
        if needle in df.index:
            hits = df.loc[[needle]]
        else:
            mask = df.astype(str).eq(needle).any(axis=1)
            hits = df.loc[mask]

        if len(hits) > 0:
            vals = pd.to_numeric(hits[ATTACK_VALUE_COL], errors="
                coerce")
            n_attacks += int((vals == 1).sum())

    out.append({"key": key, "original_name": original_name, "
        n_attacks": n_attacks})

return pd.DataFrame(out)

```

Literaturverzeichnis

- [1] B. Boswell, S. Barrett, S. Rajaganapathy, G. Dorai, and M. Qiu. FLARE: Feature-based lightweight aggregation for robust evaluation of IoT intrusion detection. *arXiv preprint arXiv:2504.15375*, 2025. Schlägt FLARE vor, eine leichtgewichtige Feature-Aggregationstechnik für IoT-IDS, die mittels zeitbasierter Sliding Windows statistische Kennzahlen wie Flussrate, Paketgrößen, Inter-Arrival-Zeiten und Entropie aggregiert, um kurzlebige Angriffsmuster wie DoS und DDoS zuverlässig zu erkennen.
- [2] CelerData. What is parquet file format?, 2024. Zugriff am: 04. Mai 2026. Erklärt die Architektur des Parquet-Formats, insbesondere die Vorteile der spaltenorientierten Speicherung und Kompression.
- [3] H. Debar and A. Wespi. Aggregation and correlation of intrusion-detection alerts. 2212:85–103, 2001. Beschreibt einen Aggregations- und Korrelationsalgorithmus für Intrusion-Detection-Alerts, der Duplikate und Folgewarnungen erkennt sowie Alerts in Situationen zusammenfasst, um dem Operator eine kondensierte Sicherheitsübersicht zu geben.
- [4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. 2019. Führt BERT ein, ein bidirektionales Repräsentationsmodell, das durch Masked Language Modeling (MLM) und Next Sentence Prediction (NSP) tiefen Kontext aus beiden Richtungen gleichzeitig lernt und damit neue Bestleistungen in elf NLP-Aufgaben erzielte.
- [5] M. K. Hasan, M. A. Alam, S. Roy, A. Dutta, M. T. Jawad, and S. Das. Missing value imputation affects the performance of machine learning: A review and analysis of the literature 2010–2021. *Informatics in Medicine Unlocked*, 27, 2021. Systematischer Review von 191 Artikeln (2010–2021) zu Missing Value Imputation (MVI) mittels der PRISMA-Methode.
- [6] M. Heigl, E. Weigelt, A. Urmann, D. Fiala, and M. Schramm. Exploiting the outcome of outlier detection for novel attack pattern recognition on streaming data. *Electronics*, 10(17):2160, 2021. Präsentiert das SOAAPR-Framework, das Ausreißererkennungsergebnisse auf Streaming-Daten auswertet und daraus neuartige Angriffsmuster extrahiert.
- [7] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. 97, 2019. Stellt den Set Transformer vor, der Self-Attention zur Kodierung von Element-Interaktionen in Mengen nutzt und durch Inducing-Point-Methoden die quadratische Komplexität von Self-Attention auf lineare Komplexität reduziert.
- [8] Z. Liu, L. Wang, B. Huang, H. Liu, H. Shi, P. Yang, and H. Dong. Pooling operations in deep learning: From invariable to variable. *BioMed Research International*, 2022. Gibt einen umfassenden Überblick über Pooling-Operationen im Deep Learning, kategorisiert sie nach invariablem und variablem Pooling-Bereich sowie -Kernel, und diskutiert deren Eigenschaften und Einsatzbereiche.
- [9] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann. E-GraphSAGE: A graph neural network based intrusion detection system for IoT. 2022. Schlägt E-GraphSAGE vor, ein GNN-basiertes Netzwerk-Intrusion-Detection-System, das sowohl Kanten-Features als

auch topologische Informationen aus Netzwerkflussdaten für die Erkennung von Angriffen in IoT-Netzwerken nutzt.

- [10] J. Norstad. An introduction to utility theory. 1999. Erläutert die mathematischen Grundlagen von Nutzenfunktionen, insbesondere die Konzepte der Nicht-Sättigung und Risikoaversion. Das Dokument führt die negative Exponential-Nutzenfunktion zur Modellierung konstanter absoluter Risikoaversion (CARA) ein und zeigt, wie mittels positiver affiner Transformationen konsistente Präferenzstrukturen skaliert werden können.
- [11] PyTorch Contributors. *MultiheadAttention — PyTorch 2.4 Documentation*. Meta AI, 2024. Zugriff am: 13. April 2026. Offizielle PyTorch-Dokumentation der `torch.nn.MultiheadAttention`-Klasse. Implementiert Multi-Head Attention nach Vaswani et al. (2017) und ermöglicht es Modellen, Informationen aus verschiedenen Repräsentationsunterräumen gemeinsam zu gewichten. Der Parameter `key_padding_mask` erlaubt das gezielte Ignorieren von Padding-Positionen bei der Attention-Berechnung.
- [12] M. T. Pérez-Maldonado, J. Bravo-Castillero, R. Mansilla, and R. O. Caballero-Pérez. New sigmoid curves: Beyond the traditional logistic models. *Revista Cubana de Física*, 42(44), 2025. Definiert S-förmige oder sigmoidale Kurven als Lösungen autonomer Differentialgleichungen erster Ordnung.
- [13] M. Sölch, A. Akhundov, P. van der Smagt, and J. Bayer. On deep set learning and the choice of aggregations. *arXiv preprint arXiv:1903.07348*, 2019. Untersucht Aggregationsfunktionen als zentralen Baustein von Deep-Set-Architekturen. Die Autoren zeigen empirisch, dass die Wahl der Aggregation (z. B. Sum, Mean, Max) die Modellperformance, Hyperparameter-Sensitivität und Generalisierung auf abweichende Eingabegrößen erheblich beeinflusst. Als Alternative zu fixen Aggregationen werden lernbare rekurrente Aggregationsfunktionen eingeführt, die robustere und konsistentere Ergebnisse liefern.
- [14] R. Taylor-Davies. How good is parquet for wide tables (machine learning workloads) really?, 2023. Zugriff am: 04. Mai 2026. Analysiert die Performance von Apache Parquet bei Tabellen mit hoher Spaltenanzahl und bewertet den Metadaten-Overhead kritisch.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. Führt die Transformer-Architektur ein, die ausschließlich auf Attention-Mechanismen basiert und Rekurrenz sowie Faltungen vollständig ersetzt, und erzielt State-of-the-Art-Ergebnisse bei maschineller Übersetzung.
- [16] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. 2018. Präsentiert Graph Attention Networks (GATs), die Masked Self-Attention nutzen, um Knoten-Repräsentationen in Graphen zu lernen, ohne kostspielige Matrixoperationen oder Vorabkenntnis der Graphstruktur zu benötigen.
- [17] O. Vinyals, S. Bengio, and M. Kudlur. Order matters: Sequence to sequence for sets. 2016. Zeigt, dass die Reihenfolge von Eingabe- und Ausgabedaten das Lernen signifikant beeinflusst, und schlägt die Read-Process-and-Write-Architektur für permutationsinvariante Mengenverarbeitung sowie eine Suchmethode für optimale Ausgabereihenfolgen vor.

- [18] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola. Deep sets. 2017. Entwickelt die DeepSets-Architektur, die permutationsinvariante Funktionen auf Mengen durch Summen-Dekomposition charakterisiert und in zahlreichen Aufgaben wie Punktwolken-Klassifikation und Populationsstatistikschätzung angewendet wird.
- [19] K. Zhang, F. Zhao, S. Luo, Y. Xin, and H. Zhu. An intrusion action-based IDS alert correlation analysis and prediction framework. *IEEE Access*, 7:150540–150551, 2019. Schlägt das IACF-Framework vor, das Alarme auf Basis intrinsischer Korrelationen zu Intrusion Actions gruppiert und mittels zeitbasierter Sequenzanalyse (TSS) zu Sessions zusammenführt, um mehrstufige Angriffe zu rekonstruieren und vorherzusagen.
- [20] X. Zhang, X. Miao, and M. Xue. A reputation-based approach using consortium blockchain for cyber threat intelligence sharing. *Security and Communication Networks*, 2022. Schlägt ein blockchain-basiertes CTI-Sharing-Modell vor, das Consortium-Blockchain mit einem verteilten Reputationsmanagementsystem kombiniert.

Generative KI-Werkzeuge, die in der Arbeit verwendet wurden.

Kapitel	KI-Tool	Version	Anwendung	Erklärung/Kommentar
1.0–9.3	Perplexity AI Gemini AI	2026	Sprachliche Überarbeitung	Der generierte Output wurde ausschließlich zur Überprüfung und Optimierung von Grammatik, Rechtschreibung und Lesbarkeit verwendet. Sämtliche fachlichen Inhalte, Analysen, Interpretationen und wissenschaftlichen Schlussfolgerungen stammen vollständig von mir.

Eidesstattliche Erklärung

Ich versichere hiermit wahrheitsgemäß, die Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt, die wörtlich oder inhaltlich übernommenen Stellen als solche kenntlich gemacht und die Satzung der Ostbayerischen Technischen Hochschule Regensburg zur Sicherung guter wissenschaftlicher Praxis in der jeweils gültigen Fassung beachtet zu haben. Die Arbeit ist weder von mir noch von einer anderen Person an der Ostbayerischen Technischen Hochschule Regensburg oder an einer anderen Hochschule zur Erlangung eines akademischen Grades bereits eingereicht worden.

Regensburg, den 27.05.2026

Yannic Hanel